ML-KEM × QUIC 연구 전 과정 기술 문서

개념 · 절차 · 명령어 · 데이터의 완전한 기록

ML-KEM 적용 시 전송 성능 개선을 위한 QUIC 기반 적응형 FEC 연구

2026 R&E · 강원과학고등학교

신레오 · 정도현 · 황요한 | 지도 이현미

작성일 2026-08-19

이 문서는 연구의 전 과정을 재현 가능한 수준으로 기록한다. 실행한 모든 명령과 그 의미, 배경 개념, 실패와 원인 진단, 원시 데이터와 전체 소스 코드를 담았다. 확정된 사실과 추정을 구분하여 표기하며, 추정에는 반드시 '추정' 또는 '미검증'을 붙였다.

목차

제 0 부 연구 개요와 결과 요약
제 1 부 배경 개념 사전 — 이 연구를 읽는 데 필요한 모든 개념
제 2 부 실험 환경 구축 — 서버, conda, 그리고 세 번의 네트워크 시도
제 3 부 1 단계 · ML-KEM 자체 성능 측정 (liboqs)
제 4 부 2 단계 · TLS/TCP 실험 — 측정기 제작과 두 번의 스윕
제 5 부 2 단계 · QUIC 실험 — 패킷 계측과 가설 기각
제 6 부 증폭 제한의 발견 — 임계, 역전, 그리고 인과
제 7 부 선행연구 9 편 대조
제 8 부 명령어 사전 — 사용한 모든 명령과 그 의미
제 9 부 함정 · 한계 · 향후 과제
부록 A 소스 코드 전문 (7 개 파일, 947 줄)
부록 B 데이터 전문 — TLS 스윕 370 행 · 증폭 실험 원시 출력
부록 C 산출물 목록

제 0 부

연구 개요와 결과 요약

0.1 연구 질문의 변천

이 연구는 하나의 질문에서 출발해 다른 질문으로 끝났다. 출발점은 계획서의 질문이었다: "ML-KEM 때문에 커진 핸드셰이크는 패킷 손실에 취약해지는가, 그렇다면 FEC 로 복구할 수 있는가." 실측은 이 질문의 전제를 기각했고, 대신 새 질문을 낳았다: "QUIC 의 증폭 제한 아래에서 키 교환 방식은 추가 왕복 임계를 어떻게 움직이는가."

이 문서는 그 전환이 어떻게 일어났는지를 명령어 수준까지 추적한다. 결과만 보면 우연한 발견처럼 보이지만, 실제로는 (1) 가설 검증 실패의 정직한 기록 → (2) "그러면 QUIC 고유의 제약은 무엇인가"라는 후속 질문 → (3) RFC 조문에서 도출한 새 가설 → (4) 인과 분리 실험이라는 연쇄였다.

0.2 핵심 결과 여섯 가지

#	결과	근거 위치
1	ML-KEM 전 연산이 0.04 ms 이하 — 병목은 연산이 아니라 전송	제 3 부
2	TLS/TCP 에서 ML-KEM-1024 만 손실 하 실패율 유의 증가(0.620%, z=+6.46). 512·768·하이브리드는 X25519 와 통계적으로 구분 불가	제 4 부
3	QUIC 무손실에서 PQC 고정 페널티 +1.18 ms. 손실 조건에서는 PTO 율·백오프 모두 유의차 없음 → 계획서 가설 H1 기각	제 5 부
4	★ 추가 왕복(1→2 RTT) 임계가 KEM 에 따라 다름: X25519 3,300~3,400 B vs 하이브리드 5,900~6,000 B. 그 사이 구간에서 PQC 가 정확히 1 RTT 빠름(격차=RTT 의 98~99%)	제 6 부
5	★ 원인은 알고리즘이 아니라 송신 바이트 수 — KEM 고정 후 패딩만 조작해도 임계 이동(패딩 인과 실험). 교환비 약 1:3.8	제 6 부
6	현재 웹 인증서 실측 분포(Tranco 75k)의 p50=4,032 B·p95=5,609 B 가 역전 구간 내부 — 발견이 현행 배치 조건에 적용됨	제 6 부

0.3 측정 규모

실험	조건	조건당 반복	총 시행
1 단계 연산 벤치마크	9 (3 알고리즘 × 3 연산)	10,000	90,000 연산
TLS 1 차 스윕	120	200~800	약 80,000 핸드셰이크
TLS 2 차 스윕	250	250~2,000	약 340,000 핸드셰이크
QUIC 손실 스윕	20	2,000	40,000 핸드셰이크
증폭 임계·패딩 스윕	136	20~200	약 6,000 핸드셰이크
재현성 검증	4~6	100~200 × 2~3 회	약 2,400 핸드셰이크

합계 약 47 만 회 시행. 모든 수치 데이터는 부록 B 와 엑셀 산출물에 원본이 있다.

제 1 부

배경 개념 사전

연구 본문(제 2~6 부)을 읽는 데 필요한 개념을 모두 여기 모았다. 각 항목은 정의 → 왜 이 연구에 필요한가 순으로 서술한다.

1.1 양자 위협과 후양자 암호(PQC)

현대 웹 암호화의 키 교환은 타원곡선(X25519 등)과 RSA 에 의존한다. 이들의 안전성은 이산로그·소인수분해가 어렵다는 가정 위에 있는데, Shor 알고리즘(1994)은 충분히 큰 양자컴퓨터에서 두 문제를 모두 다항 시간에 푼다. 즉 대규모 양자컴퓨터가 실현되는 순간 현재의 키 교환은 전부 무력화된다.

문제는 시점이 '실현 이후'가 아니라는 것이다. HNDL(Harvest Now, Decrypt Later) — 공격자가 오늘의 암호화 트래픽을 통째로 저장해 두었다가, 미래에 양자컴퓨터로 복호화하는 전략이다. 서명은 위조 시점에만 문제가 되므로 나중에 교체해도 되지만, 키 교환은 과거 트래픽까지 소급해서 뚫린다. 그래서 전 세계 전환이 키 교환부터 시작되었다.

NIST 는 2016 년 공모를 열어 2022 년 알고리즘을 선정했고, 2024 년 8 월 FIPS 203 으로 ML-KEM(Module-Lattice-based KEM, 구 명칭 CRYSTALS-Kyber)을 확정했다. 격자 문제(Module-LWE)에 기반하며, 양자컴퓨터로도 효율적 공격이 알려져 있지 않다. 대가는 크기다 — 고전 키 교환의 공개값이 32 바이트인 반면 ML-KEM 은 800~1,568 바이트로, 이 크기 차이가 본 연구 전체의 출발점이다.

1.2 KEM 이란 — DH 와의 차이

전통적 Diffie-Hellman 은 양쪽이 공개값을 교환하고 각자 같은 비밀을 계산하는 대칭 구조다. KEM(Key Encapsulation Mechanism)은 비대칭 구조로, 세 연산으로 정의된다.

```
keygen()        → (ek, dk)      캡슐화키(공개) · 복호화키(비밀) 생성
encaps(ek)      → (ct, ss)      상대 공개키로 암호문과 공유비밀 생성
decaps(dk, ct)  → ss            암호문을 풀어 같은 공유비밀 복원
```

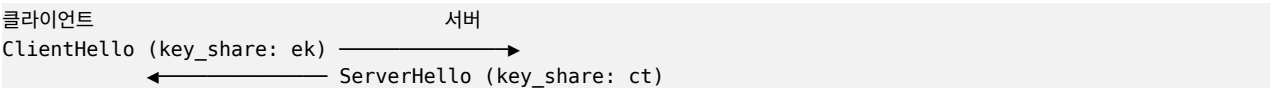
TLS 1.3 의 key_share 확장에 그대로 대응된다: 클라이언트가 ClientHello 에 ek 를 실어 보내고, 서버가 encaps 를 수행해 ct 를 돌려주며, 양쪽이 ss 로부터 세션 키를 유도한다. 따라서 네트워크에 실리는 것은 ek(상행)와 ct(하행)이고, 이 두 크기가 전송 비용을 결정한다.

1.3 ML-KEM 의 세 보안수준과 하이브리드

알고리즘	ek (B)	ct (B)	공유비밀	비고
X25519 (고전, 기준)	32	32	32 B	타원곡선 DH
ML-KEM-512	800	768	32 B	NIST 보안수준 1
ML-KEM-768	1,184	1,088	32 B	수준 3 · 권장 기본
ML-KEM-1024	1,568	1,568	32 B	수준 5
X25519MLKEM768 (하이브리드)	1,216	1,120	결합	32+1,184 연접
SecP256r1MLKEM768 (하이브리드)	1,249	1,153	결합	65+1,184 연접

하이브리드는 고전과 후양자 키 교환을 동시에 수행하고 두 비밀을 해시로 결합한다. 어느 한쪽이 깨져도 안전하다는 중첩 방어가 목적이다 — ML-KEM 은 표준화된 지 얼마 되지 않아 검증 기간이 짧고, X25519 는 양자에 취약하므로, 전환기에는 둘을 겹치는 것이 합리적이다. TLS 1.3 규격상 하이브리드 키셰어는 두 값의 단순 연접이라 크기가 정확히 합이 된다. 현재 Chrome·Cloudflare·OpenSSL 이 X25519MLKEM768 을 기본값으로 채택했으며, 순수 ML-KEM 의 실배치는 드물다. 본 연구의 QUIC 실험이 하이브리드로 수행된 것은 이 배치 현실과 Go 표준 라이브러리의 지원 범위(제 5 부) 때문이다.

1.4 TLS 1.3 핸드셰이크 구조



```
            ←──────────── EncryptedExtensions
            ←──────────── Certificate  ★ 서버 첫 비행의 대부분
            ←──────────── CertificateVerify
            ←──────────── Finished
Finished ──────────────────►        (이후 애플리케이션 데이터)
```

TLS 1.3 은 1 왕복(1-RTT)에 키 합의와 인증을 끝낸다. 서버가 보내는 것 중 압도적으로 큰 것이 Certificate 메시지, 즉 인증서 체인이다. 웹 실측(Tranco 75,000 개)에서 체인의 중앙값은 4,032 바이트, 95 백분위는 5,609 바이트로, 키셰어(수백~천 바이트대)보다 크다. 서버 응답의 크기를 좌우하는 것은 키 교환이 아니라 인증서라는 사실이 제 6 부 실험 설계의 근거가 된다.

한 가지 통제 사항: TLS 에는 세션 재개(session resumption)가 있어 이전 연결의 비밀을 재사용하면 키 교환을 건너뛴다. 재개가 일어나면 측정이 무의미해지므로 본 연구는 모든 측정에서 재개를 차단했다(제 4 부 4.2).

1.5 TCP+TLS 와 QUIC 의 차이

TCP 위의 TLS 는 두 단계다 — TCP 3-way 핸드셰이크(1 RTT)를 먼저 끝내고 그 위에서 TLS(1 RTT)를 수행하므로 최소 2 RTT 가 든다. QUIC(RFC 9000)은 전송 계층과 TLS 1.3 을 하나로 합쳐 UDP 위에서 동작하며, 연결 수립과 키 합의를 동시에 진행해 1 RTT 로 끝낸다.

본 연구와 직결되는 구조적 차이는 세 가지다. 첫째, QUIC 은 UDP 데이터그램 단위로 동작하므로 패킷 경계가 명확하고 계측이 데이터그램 수·크기로 떨어진다(제 5 부 countConn). 둘째, QUIC 은 커널이 아니라 사용자공간 라이브러리(quic-go 등)로 구현되어 구현별 정책 차이가 크다. 셋째 — 가장 중요한 차이 — QUIC 에는 TCP 에 없는 증폭 제한 규정(1.9)이 있어, 주소 검증 전 서버 송신량이 클라이언트 송신량에 묶인다. TCP 의 대응물인 초기 혼잡 윈도우(1.7)는 고정값이라는 점이 제 6 부 결론(역전은 QUIC 고유)의 근거다.

1.6 MTU · MSS · 데이터그램

MTU(Maximum Transmission Unit)는 링크가 한 번에 나를 수 있는 최대 바이트로, 이더넷 표준은 1,500 이다. TCP 는 여기서 IP·TCP 헤더 40 바이트를 뺀 MSS 1,460 바이트 단위로 데이터를 쪼갠다(세그먼트). QUIC 은 UDP/IP 헤더 28 바이트를 뺀 최대 1,472 바이트의 데이터그램을 쓸 수 있으나, quic-go 의 기본 데이터그램 크기는 1,280 바이트다 — IPv6 최소 MTU 에 맞춘 보수적 값이며, RFC 9000 이 요구하는 최소 1,200 보다 여유가 있다.

이 개념이 중요한 이유: loopback 인터페이스의 기본 MTU 는 65,536 이다. 이 값에서는 3,000 바이트짜리 ClientHello 도 한 세그먼트에 다 들어가므로, 키 크기에 따른 분할 차이 — 본 연구가 보려는 현상 — 가 아예 발생하지 않는다. 1 차 TLS 스윕이 무효가 된 원인이자(제 4 부 4.5), 실험 환경에서 MTU 1,500 강제가 통제변수 1 순위인 이유다.

1.7 TCP 초기 혼잡 윈도우(IW10)

TCP 서버는 클라이언트의 ACK 를 받기 전에도 일정량을 먼저 보낼 수 있는데, 그 한도가 초기 혼잡 윈도우다. RFC 6928 이후 관행은 10 MSS, 약 14.6 KB 다. 즉 TLS 서버 첫 비행(인증서 포함)이 14.6 KB 이내면 왕복 추가 없이 한 번에 나간다.

이 값이 두 곳에서 등장한다. 첫째, 1 차 TLS 스윕에서 KEM 간 격차가 RTT 와 무관하게 일정했던 이유 — 최대 3 KB 인 키셰어가 IW 에 통째로 흡수되어 추가 왕복이 생기지 않았다(제 4 부). 둘째, 선행연구 Chou & Cao(2026)가 보고한 TTFB 도약 임계 10 KB·40 KB 가 바로 이 윈도우(IW10 과 그 확장) 경계다. TCP 의 이 한도는 클라이언트가 무엇을 보내든 고정이라는 점이 QUIC 증폭 예산(1.9)과의 결정적 차이다.

1.8 QUIC Initial 패킷과 1,200 바이트 최소 패딩

RFC 9000 §14.1 은 클라이언트의 Initial 패킷을 담은 UDP 데이터그램이 최소 1,200 바이트가 되도록 요구한다. 내용이 그보다 작으면 PADDING 프레임(값 0x00 의 반복)으로 채워야 한다. 목적은 두 가지다 — 경로가 최소 크기의 패킷을 나를 수 있음을 보장하고(PMTU), 작은 요청으로 큰 응답을 끌어내는 증폭 공격의 이득을 줄이는 것.

본 연구 관점에서 핵심은 이것이다: 실측에서 X25519 클라이언트의 첫 비행은 2,560 B(1,280 B 데이터그램 2 개)인데 ClientHello 실내용은 수백 바이트에 불과하다. 약 2 KB 가 규정상 어쩔 수 없이 실려 가는 패딩이며, 이 '버려지는 공간'이 제 6 부의 패딩 인과 실험과 FEC 재정의(패딩을 복구 심볼로 대체)의 물리적 토대다.

1.9 증폭 공격과 3 배 제한 (RFC 9000 §8.1)

```
공격자: 출발지 IP를 피해자 주소로 위조한 작은 요청 ──▶ 서버
서버:   큰 응답을 '피해자'에게 전송  (요청의 수십 배 크기)
결과:   공격자는 작은 대역폭으로 피해자에게 큰 트래픽을 반사
```

UDP 기반 프로토콜의 고전적 취약점이다. QUIC 의 방어는 두 겹이다. 첫째가 1.8 의 최소 1,200 B 요구(요청을 키워 증폭 이득 축소), 둘째가 §8.1 의 규정 — 서버는 클라이언트 주소가 검증되기 전까지, 그 주소로부터 받은 바이트의 3 배를 초과해 보낼 수 없다. 여기서 '받은 바이트'는 UDP 데이터그램 전체이므로 패딩도 포함된다.

예산이 바닥나면 서버는 전송을 멈추고 클라이언트의 다음 패킷(주로 ACK)을 기다린다. ACK 가 도착하면 주소가 검증되어 제한이 해제되지만, 그 대기가 정확히 왕복 한 번이다. 대안으로 Retry 패킷(토큰 발급)이 있으나 모든 연결에 1 RTT 를 고정 추가하므로 실서비스가 기피한다.

이 조문에서 본 연구의 가설이 직접 도출된다: 예산 = 클라이언트 송신 × 3 이므로, 키셰어가 커서 클라이언트가 더 보내는 후양자 키 교환은 서버 예산을 키운다. 인증서가 '고전 예산은 넘지만 후양자 예산은 안 넘는' 크기라면 — 후양자 쪽만 1-RTT 를 유지한다.

1.10 손실 감지 — PTO 와 지수 백오프

QUIC 은 TCP 의 RTO 대신 PTO(Probe Timeout)를 쓴다. ACK 가 와야 할 시간이 지나도 오지 않으면 타이머가 만료되고, 확인되지 않은 데이터를 탐침으로 재전송한다.

```
PTO = smoothed_RTT + max(4 × rttvar, granularity) + max_ack_delay
만료마다 PTO ← PTO × 2   (지수 백오프)
```

제 5 부의 손실 실험에서 종속 지표가 바로 이것이다 — 핸드셰이크 중 PTO 가 발생한 연결의 비율(PTO 율)과, 한 연결에서 백오프가 몇 단까지 갔는가(백오프 깊이). 계획서 가설 H1 은 "PQC 는 패킷이 많아 PTO 를 더 자주 겪을 것"으로 조작화되었다.

1.11 netem 과 Gilbert-Elliott 손실 모델

netem 은 리눅스 트래픽 제어(tc)의 큐 규칙(qdisc)으로, 인터페이스 송신 큐에 지연·손실·재정렬을 인위적으로 주입한다. 본 연구에서 쓴 세 형식:

```
tc qdisc add dev v0 root netem delay 25ms          # 고정 지연 25 ms
tc qdisc add dev v0 root netem loss 5%             # 균일 손실: 매 패킷 독립 5%
tc qdisc add dev v0 root netem loss gemodel 5% 50%  # 버스트 손실(GE 모델)
```

Gilbert-Elliott 모델은 2 상태 마르코프 연쇄다 — good 상태에서 p 확률로 bad 로 넘어가고, bad 에서 r 확률로 good 으로 복귀하며, bad 동안 패킷이 유실된다. gemodel 5% 50%는 p=5%, r=50%로 평균 버스트 길이 1/r = 2 패킷을 뜻한다. 실제 무선망의 연속 유실을 근사하는 표준적 방법이나, 본 연구의 p·r 값 자체는 문헌 근거 없이 정한 것이라 한계로 기록했다(제 9 부). 또 netem 은 난수 시드를 고정할 수 없어 같은 조건도 실행마다 손실 패턴이 다르다 — 반복 실행으로 완화했다.

1.12 통계 개념

백분위(percentile)

정렬된 표본에서 p% 위치의 값. p50 은 중앙값, p95·p99 는 꼬리다. 지연 측정에서 평균은 극단값에 끌려가므로 p50 을 대표값으로, 꼬리 악화 여부는 p95·p99 로 본다. hct_v2.c 의 pct() 함수는 선형 보간 방식을 쓴다 — 위치가 정수가 아니면 이웃 두 값을 비율로 섞는다.

표본 표준편차

산포의 척도. 분산을 n 이 아니라 n−1 로 나눈다(표본이 모집단보다 산포를 과소평가하는 것을 보정). 측정 안정성 판단과 워밍업 효과 확인에 썼다.

이표본 비율 z-검정

두 그룹의 실패율 차이가 우연인지 판정한다. 합동비율 $\hat{p}$ = (실패 1+실패 2)/(n1+n2)로 표준오차를 만들고:

```
z = (p1 − p2) / sqrt( p̂(1−p̂) × (1/n1 + 1/n2) )
```

|z| > 1.96 이면 5% 수준에서, > 3.29 면 0.1% 수준에서 유의하다. 2 차 TLS 스윕에서 ML-KEM-1024 의 z=+6.46 은 우연으로 볼 수 없는 차이(p<0.001)이고, 768 의 z=+0.82 는 우연 범위다. 여러 그룹을 동시에 비교하므로 경계선 값은 보수적으로 읽어야 한다.

의사반복(pseudo-replication)과 워밍업

한 실행 안의 n 회 반복은 같은 프로세스·같은 netem 난수열을 공유하므로 완전히 독립인 표본이 아니다. n 을 키우는 것과 별개로, 시점을 달리한 독립 실행 반복이 재현성의 근거가 된다 — 증폭 임계 검증을 3 회 독립 실행으로 수행한 이유다(제 6 부). 워밍업은 초기 수십 회를 통계에서 제외하는 절차로, CPU 캐시·분기예측기·JIT 안정화 전의 이상 고지연을 걸러낸다.

제 2 부

실험 환경 구축

2.1 하드웨어와 권한 제약

항목	값
호스트	kebap-server (교내 연구 서버)
CPU / 메모리	Intel Xeon Gold 5218R @ 2.10 GHz 40 코어 / 125 GiB
OS / 커널	Ubuntu 24.04.4 LTS / 7.0.0-28-generic
권한	일반 사용자 — sudo 없음
작업 경로	~/Desktop/workspace/34/1311 신레오 (폴더명에 공백 포함)

sudo 가 없다는 사실이 설계 전체를 지배했다. 네트워크 에뮬레이션(tc·netem·네임스페이스)은 본래 root 작업이다. 해법은 비특권 사용자 네임스페이스 — unshare 로 격리 공간을 만들면 그 안에서만 root 가 된다(2.4). 패킷 캡처도 tcpdump 대신 프로그램 내부 계측으로 대체했다(제 5 부).

폴더명의 공백('1311 신레오')은 셸 스크립트에서 경로가 단어 두 개로 쪼개지는 사고를 반복해서 일으켰다. 표준 대처는 두 가지 — 경로를 항상 따옴표로 감싸거나, 실행 파일을 공백 없는 /tmp/로 복사해 쓰는 것. 본 연구는 후자를 관례로 삼았다(cp qamp /tmp/qamp).

2.2 conda 환경 oreo_34

시스템 기본 OpenSSL 은 3.0.13 으로 ML-KEM 이 없다. 모든 작업은 conda 가상환경 안에서 한다 — PATH 와 라이브러리 검색 경로를 환경 내부로 교체하는 것이 activate 의 실체다.

```
conda activate oreo_34        # 모든 세션의 첫 명령
openssl version               # OpenSSL 3.5.6 확인 (3.5 부터 ML-KEM 네이티브)
go version                    # go1.26.5
python3 -c 'import oqs; print(oqs.oqs_version())'   # liboqs 0.15.0
```

도구	버전	용도
OpenSSL	3.5.6	TLS 실험 · ML-KEM 5 종 그룹
liboqs-python	0.15.0	1 단계 알고리즘 벤치마크
Python	3.12.13	측정 스크립트·분석
Go	1.26.5	QUIC 도구 빌드
quic-go	v0.61.0	QUIC 프로토콜 구현
iproute2 (ip·tc)	6.1.0	네임스페이스·netem
Rust (rustup)	1.97.1	quiche 교차검증 시도(제 9 부)

2.3 네트워크 격리 — 세 번의 시도

네트워크 조건(RTT·손실·MTU)을 통제하려면 격리된 링크가 필요하다. 최종 구성에 도달하기까지 두 번 실패했고, 각 실패가 중요한 교훈을 남겼으므로 전 과정을 기록한다.

시도 1 — loopback + netem (실패)

```
unshare -Urn bash <<'INNER'
ip link set lo up
ip link set lo mtu 1500        # ★ 문제의 명령
tc qdisc add dev lo root netem delay 25ms loss 2%
INNER
```

가장 단순한 구성 — 서버·클라이언트를 같은 호스트에서 127.0.0.1 로 잇고 lo 에 netem 을 건다. 그런데 lo 의 MTU 변경 명령은 종료 코드 0 을 반환하면서 실제로는 적용되지 않는다. 커널이 조용히 무시한다. 오류가 없으니 발견도 늦었다 — 1 차 TLS 스윕 120 조건을 전부 돌린 뒤에야 이상을 감지했다(제 4 부 4.5).

```
cat /sys/class/net/lo/mtu      # → 65536   (1500 이 아님을 확인한 진단 명령)
```

교훈: 종료 코드 0 은 '적용됨'이 아니다. 설정 명령은 반드시 별도 조회로 검증한다.

시도 2 — 단일 네임스페이스 + veth (실패)

MTU 가 진짜로 적용되는 가상 이더넷(veth) 쌍으로 바꿨다. 그런데 veth 양끝(v0·v1)을 같은 네임스페이스에 두면, 커널이 '어차피 내 주소'라며 loopback 경로로 우회한다 — veth 를 만들었지만 트래픽은 lo 로 흐른 것이다.

```
ip route get 10.9.0.2          # → ... dev lo ...   (우회를 확인한 진단 명령)
```

시도 3 — 중첩 네임스페이스 + veth (성공 · 최종 구성)

```
unshare -Urnm bash -c '
mount -t tmpfs none /run 2>/dev/null; mkdir -p /run/netns
ip netns add peer
ip link add v0 type veth peer name v1
ip link set v1 netns peer
ip addr add 10.9.0.1/24 dev v0; ip link set v0 mtu 1500; ip link set v0 up
ip netns exec peer sh -c "ip addr add 10.9.0.2/24 dev v1; ip link set v1 mtu 1500; ip link set v1 up"
tc qdisc add dev v0 root netem delay 25ms
ip netns exec peer tc qdisc add dev v1 root netem delay 25ms
'
```

바깥 네임스페이스 안에 또 하나(peer)를 만들고 veth 한쪽을 그리로 보낸다. 커널 관점에서 두 개의 독립 호스트가 케이블로 연결된 것과 같아, MTU·netem 이 실제 네트워크처럼 동작한다. 각 명령의 의미:

명령	의미
unshare -U -r -n -m (합쳐서 -Urnm)	-U 새 사용자 네임스페이스 / -r 현재 사용자를 그 안의 root 로 매핑 / -n 네트워크 격리 / -m 마운트 격리. 이 조합이 sudo 없이 root 권한 네트워크 작업을 가능하게 한다
mount -t tmpfs none /run	ip netns 가 /run/netns 밑에 마운트 지점을 만드는데 원래 /run 은 이 네임스페이스에서 쓸 수 없다. 메모리 파일시스템(tmpfs)으로 덮어 쓰기 가능하게 만든다
ip netns add peer	'peer'라는 이름의 두 번째 네트워크 네임스페이스 생성
ip link add v0 type veth peer name v1	가상 이더넷 케이블 생성 — 항상 쌍(v0↔v1)으로 만들어지며 한쪽에 넣은 패킷이 반대쪽에서 나온다
ip link set v1 netns peer	v1 을 peer 네임스페이스로 이동 → 양끝이 서로 다른 '호스트'에 놓임 (시도 2 의 우회를 차단)
ip addr add / ip link set mtu / up	주소 부여, MTU 1500 강제, 인터페이스 활성화
ip netns exec peer CMD	peer 네임스페이스 안에서 CMD 실행 — 서버는 항상 이 안에서 기동
tc qdisc ... netem delay 25ms (양쪽)	각 방향에 25 ms → 왕복 50 ms. 한쪽에만 50 ms 를 걸면 편도만 느린 비대칭이 되므로 반드시 절반씩 대칭으로 건다

구성 검증

```
cat /sys/class/net/v0/mtu                    # 1500 확인
ping -c 20 -i 0.2 10.9.0.2 | tail -1         # rtt avg = 50.068 ms 확인
```

ping 의 -c 20 은 20 회, -i 0.2 는 0.2 초 간격, 마지막 줄이 min/avg/max 통계다. 평균 50.068 ms — 목표 50 ms 에 오차 0.1% 이내로 netem 이 정확히 적용됨을 확인했다.

2.4 실행 규율 — 측정을 지키는 습관들

규율	명령 형태	이유
CPU 고정	taskset -c 0-3 CMD	40 코어 사이를 옮겨 다니면 캐시 미스로 분산이 커진다. 4 개 코어에 고정
장기 실행	nohup bash run.sh > log 2>&1 &	로그아웃해도 계속. 표준출력·오류를 로그로. & 는 백그라운드
진행 감시	tail -f log	파일 끝을 실시간 추적(-f). Ctrl+C 로 감시만 중단(작업은 계속)
시간 상한	timeout 120 CMD	120 초 후 강제 종료. 무한 대기를 원천 차단 — 새 코드 첫 실행의 필수 수칙

프로세스 정리	SRV=$!; ... kill $SRV	$!는 직전 백그라운드 PID. pkill -f 패턴은 부모 셸까지 죽인 사고가 있어 PID 지정 방식으로 통일
포트 분리	조건마다 다른 포트	이전 조건의 서버 프로세스 잔존이 다음 측정을 오염시키는 것을 방지(제 6 부 이상치의 유력 원인)

제 3 부

1 단계 · ML-KEM 자체 성능 측정 (liboqs)

3.1 목적과 도구

첫 질문은 가장 기초적인 것이었다 — ML-KEM 의 세 연산(keygen·encaps·decaps)은 얼마나 걸리고, 주고받는 데이터는 얼마나 큰가. 네트워크를 붙이기 전에 알고리즘 자체의 비용을 분리해서 재야, 이후 실험에서 관측되는 차이가 '연산 때문인지 전송 때문인지'를 가릴 수 있다.

도구는 liboqs-python 0.15.0 — Open Quantum Safe 프로젝트의 C 라이브러리를 파이썬으로 감싼 것이다. 네트워크·TLS 와 무관하게 순수 알고리즘만 실행한다. 선행연구 Sakk 외(2026)가 동일 라이브러리·동일 버전을 사용했다는 점이 나중에 교차검증을 가능하게 했다(3.4).

3.2 측정 방법

```
import oqs, time
N, WARM = 10000, 100
for alg in ['ML-KEM-512','ML-KEM-768','ML-KEM-1024']:
    kem = oqs.KeyEncapsulation(alg)
    for i in range(WARM + N):
        t0 = time.perf_counter_ns()
        ek = kem.generate_keypair()          # keygen 구간
        t1 = time.perf_counter_ns()
        ct, ss = kem.encap_secret(ek)        # encaps 구간
        t2 = time.perf_counter_ns()
        ss2 = kem.decap_secret(ct)           # decaps 구간
        t3 = time.perf_counter_ns()
        # i >= WARM 인 경우만 (t1-t0, t2-t1, t3-t2) 기록
```

연산당 N=10,000 회, 워밍업 100 회 제외. perf_counter_ns 는 나노초 단위 단조 시계다(벽시계 조정의 영향을 받지 않음). 위 코드는 측정 로직의 요지이며 실제 스크립트는 CSV 기록·통계 산출이 추가된 형태다. 크기는 liboqs API 가 알고리즘 상수로 반환하는 값을 그대로 읽었고, FIPS 203 규격값과 일치함을 확인했다.

3.3 결과

알고리즘	keygen (µs)	encaps (µs)	decaps (µs)	최대 표준편차
ML-KEM-512	15.6	17.8	18.9	2.7 µs
ML-KEM-768	24.1	25.9	28.2	7.4 µs
ML-KEM-1024	29.8	32.6	36.5	6.8 µs

가장 느린 조합(ML-KEM-1024 decaps)조차 36.5 µs = 0.037 ms. 네트워크 왕복(수십 ms)의 1/1000 수준이다. → 후양자 전환의 비용이 있다면 그것은 연산이 아니라 전송에서 나온다. 이 결론이 2 단계 전체의 방향을 정했다.

크기 데이터(ek/ct)는 1.3 의 표와 같다. 수준이 오를수록 ek 는 800→1,184→1,568 로 커지고, 이것이 곧 ClientHello 팽창분이다.

3.4 외부 교차검증 — Sakk 외 (2026)

Future Internet 게재 논문 Sakk 외(2026)가 같은 liboqs 0.15.0 으로 독립 측정한 수치와 대조했다. 9 개 항목(3 알고리즘 × 3 연산) 중 8 개가 ±3 µs 이내로 일치했다. 남은 1 개 항목의 차이는 캐시 환경 차이로 추정되나 미검증이다. 서로 다른 하드웨어에서 같은 자릿수·같은 서열이 재현된 것으로, 1 단계 수치의 신뢰도를 외부적으로 뒷받침한다. 상세 대조는 산출물 '선행연구노트_Sakk2026.docx'에 있다.

2 단계 · TLS/TCP 실험 — 측정기 제작과 두 번의 스윕

4.1 왜 측정기를 직접 만들었나

기성 도구 두 가지를 먼저 검토하고 기각했다. openssl s_client 는 연결 1 회마다 프로세스를 새로 띄우는데, 프로세스 기동 비용(수 ms)이 측정 대상인 핸드셰이크 시간(무손실에서 수 ms)과 같은 자릿수라 신호가 잡음에 묻힌다. openssl s_time 은 반복 측정은 되지만 협상 그룹 검증·타임아웃·연결별 CSV 가 없다. 특히 타임아웃 부재는 치명적이다 — 손실 환경에서 한 연결이 멈추면 전체가 멈춘다.

그래서 C 로 직접 작성했다(hct_v2.c, 120 줄, 전문은 부록 A). 한 프로세스 안에서 소켓 생성→TLS 핸드셰이크→종료를 수천 번 반복하며, 반복 간 비용이 마이크로초 수준이라 측정을 오염시키지 않는다.

4.2 hct_v2.c 의 여덟 가지 설계 결정

#	결정	코드	이유
1	단조 시계 사용	clock_gettime(CLOCK_MONOTONIC,...)	벽시계는 NTP 조정으로 역행할 수 있다. 구간 측정은 단조 시계가 원칙
2	Nagle 비활성	setsockopt(...,TCP_NODELAY,...)	작은 패킷을 모아 보내는 지연(수십 ms)이 측정에 섞이는 것을 차단
3	논블로킹 connect + poll	fcntl(...O_NONBLOCK); poll(&pfd,1,TMO)	블로킹 connect 는 실패 시 커널 기본값(수 분)을 기다린다. poll 로 상한(TMO=5,000 ms)을 직접 통제
4	TLS 진행 루프	SSL_ERROR_WANT_READ/WRITE 분기	논블로킹에서는 SSL_connect 가 여러 번 호출된다. '아직 읽을/쓸 차례'를 poll 이벤트로 기다렸다 재개
5	세션 재개 차단	SSL_SESS_CACHE_OFF + SSL_OP_NO_TICKET	재개되면 키 교환을 건너뛰어 측정이 무의미(1.4)
6	협상 그룹 검증	SSL_group_to_name(SSL_get_negotiated_group())	요청한 그룹과 실제 협상 그룹이 다르면 경고. OpenSSL 3.5 서버 기본값이 하이브리드만 제안하는 함정(4.7) 때문에 필수
7	워밍업 분리	i < warmup 이면 통계 제외	캐시 안정화 전 이상 고지연 배제(1.12)
8	연결별 CSV	tcp_ms, tls_ms, status 열	실패를 tcp_timeout / tls_timeout / fail 로 구분 기록 — 어느 단계에서 죽었는지 사후 진단 가능

1 차 도구(hct_client.c)에는 3 번이 없었다. 그 결과 버스트 5% 조건에서 두 조건이 무한 대기로 멈췄고, 더 나쁘게는 멈춘 연결이 실패로 집계되지 않아 '실패율 0.00%'라는 그럴듯한 오답을 냈다. v2 의 타임아웃·실패 분류는 그 사고의 직접적 산물이다.

4.3 컴파일 명령과 각 플래그의 의미

```
gcc -O2 hct_v2.c -o hct_v2 \
  -I$CONDA_PREFIX/include -L$CONDA_PREFIX/lib \
  -lssl -lcrypto -lm -Wl,-rpath,$CONDA_PREFIX/lib
```

플래그	의미
-O2	최적화 2 단계 — 측정기 자체 오버헤드 축소
-I / -L	헤더·라이브러리를 conda 의 OpenSSL 3.5.6 에서 찾도록 지정 (시스템 3.0.13 회피)
-lssl -lcrypto -lm	libssl(TLS)·libcrypto(암호)·수학 라이브러리 링크
-Wl,-rpath,...	실행 파일 안에 런타임 라이브러리 경로를 각인. 이것이 없으면 실행 때 시스템 OpenSSL 을 물어 ML-KEM 그룹을 못 찾는다. LD_LIBRARY_PATH 없이도 항상 올바른 라이브러리를 쓰게 하는 장치

4.4 스윕 스크립트 run_v3.sh 의 구조

전문은 부록 A(106 줄). 구조적 장치 네 가지만 짚는다.

① 이중 히어독(OUTER/INNER). 바깥 히어독은 스크립트 파일 자체를 만들고, 안쪽 히어독은 unshare 가 띄운 셸에 격리 환경 안에서 실행할 내용을 전달한다. 안쪽으로 값을 넘기려면 export OUT BIN CERT … 로 환경변수를 내보내야 한다 — 서브셸은 export 된 변수만 상속하기 때문이다.

② 반복 수 자동 조절. 무손실 조건은 분산이 작아 ITERS 의 1/8 만, RTT 100·200 ms 조건은 회당 시간이 길어 절반만 돈다. 정밀도가 필요한 곳(손실 조건)에 시행을 집중해 총 10 시간 예산을 지키는 설계다.

③ 협상 검증 경고. 측정기 출력에서 negotiated 값을 sed 로 추출해 요청 그룹과 대조하고, 다르면 '!! 협상불일치'를 출력한다. 조용한 그룹 대체를 스윕 도중에 즉시 발견하기 위함이다.

④ 요약 CSV 집계. 측정기의 사람용 출력에서 sed -n 's/.*p50=\([0-9.]*\).*/\1/p' 형태로 수치를 뽑아 summary.csv 한 줄로 만든다. -n 은 자동 출력 억제, \(…\)는 캡처, /p 는 매칭된 줄만 출력이라는 뜻이다.

주의 — 판본 차이: 부록 A 의 run_v3.sh 초판은 netem 을 v0 한쪽에만 걸어 delay=RTT 전체를 단방향으로 주입한다. 이후 대화에서 양방향 대칭(각 RTT/2)으로 패치되어 최종 스윕과 이후 모든 QUIC·증폭 실험에 쓰였다 (2.3 의 최종 구성). 단방향은 편도 비대칭이라 실제 망과 다르기 때문이다.

4.5 1 차 스윕 — 그리고 MTU 결함의 발견

```
nohup bash run_full.sh full > sweep1.log 2>&1 &      # 밤샘 실행
tail -f sweep1.log                                   # 진행 감시
```

120 조건, 약 8 만 회. 결과의 첫인상은 '너무 깨끗하다'였다 — 모든 손실·RTT 조건에서 KEM 간 HCT 격차가 0.57~0.73 ms 로 거의 일정했고, 실패율 차이도 없었다.

격차가 RTT 와 무관하게 일정하다는 것은 추가 왕복이 어디에서도 발생하지 않았다는 뜻이다. 그렇다면 모든 KEM 의 ClientHello 가 같은 세그먼트 수로 나갔다는 의심이 선다. 진단해 보니 loopback MTU 가 65,536 그대로였다 (2.3 시도 1). MTU 65,536 에서는 3 KB 키셰어도 한 세그먼트다 — 분할 차이가 애초에 만들어질 수 없는 환경이었다.

1 차 스윕의 재해석: 손실 검증으로는 무효지만, 'MTU 가 충분히 크면(=IW 가 흡수하면) KEM 간 차이가 사실상 사라진다'는 대조군으로 살릴 수 있다. 실패한 데이터를 버리지 않고 역할을 바꿔 보존한 사례다.

4.6 2 차 스윕 — MTU 1500, 250 조건 34 만 회

veth 구성(2.3 시도 3)에서 재실행했다. 그룹별 합산 실패율:

그룹	실패율	z (X25519 대비)	판정
X25519 (기준)	0.367%	—	기준
ML-KEM-512	0.362%	−0.14	차이 없음
ML-KEM-768	0.395%	+0.82	차이 없음
ML-KEM-1024	0.620%	+6.46	$p < 0.001$ 유의
X25519MLKEM768	0.416%	+1.39	차이 없음

해석: MSS 1,460 기준으로 ML-KEM-1024(ek 1,568 B)만 ClientHello 가 확실히 2 세그먼트로 갈라진다. 세그먼트가 하나 늘면 '적어도 하나 유실' 확률이 커지는 것은 사실이나, 그 효과가 유의하게 드러난 것은 최고 보안수준뿐이다. 768(1,184 B)은 경계에 걸치는데 z=+0.82 로 잡음 범위 — 실무 권장 수준(768)과 하이브리드에서는 TCP/TLS 실패율 관점의 페널티가 관측되지 않았다.

4.7 OpenSSL 3.5 의 그룹 함정

스윕 전 예비 시험에서 순수 ML-KEM 그룹 협상이 계속 실패했다. 원인은 OpenSSL 3.5 서버의 기본 그룹 목록이 하이브리드 중심이라는 것 — 클라이언트가 MLKEM768 을 요구해도 서버가 제안하지 않으면 협상이 깨진다. 해법은 양쪽 모두 -groups 명시:

```
openssl s_server -cert c.pem -key k.pem -tls1_3 -groups MLKEM768 -accept 127.0.0.1:45577 -quiet
# -tls1_3: 버전 고정 / -groups: 키 교환 그룹 강제 / -quiet: 연결 로그 억제 / -accept: 대기 주소
```

이 함정이 hct_v2 의 설계 결정 6(협상 그룹 검증)을 낳았다. '측정이 되는 것'과 '의도한 것을 측정하는 것'은 다르다.

제 5 부

2 단계 · QUIC 실험 — 패킷 계측과 가설 기각

5.1 구현 선택과 제약

QUIC 구현으로 quic-go v0.61.0 을 선택했다. 순수 Go 라 빌드가 단순하고, Transport 에 소켓을 직접 주입하는 API 가 있어 계측 훅을 끼울 수 있으며, 실서비스 채택도 넓다. 단, quic-go 는 TLS 계층을 Go 표준 crypto/tls 에 위임하는데 이 라이브러리는 X25519MLKEM768 하이브리드만 지원한다 — 순수 ML-KEM 3 종은 QUIC 에서 측정 불가라는 본 연구 최대의 제약이 여기서 생겼다(제 9 부 한계 2).

5.2 빌드 함정 — CGO

```
go mod init quicbench && go get github.com/quic-go/quic-go@v0.61.0
CGO_ENABLED=0 go build -o qpkt qpkt.go
```

conda 환경이 CC 변수를 conda 전용 컴파일러로 지정해 두었는데 실제로는 설치되어 있지 않아, 기본 빌드가 'cgo: C compiler not found'로 실패한다. CGO_ENABLED=0 은 C 연동을 끄고 순수 Go 로만 빌드하라는 지시로, 이 프로젝트는 C 의존이 없으므로 부작용이 없다. 이후 모든 Go 빌드의 표준 접두어가 되었다.

5.3 qlog 의 실패 — 그리고 방향 전환

처음 계획은 QUIC 표준 계측 형식인 qlog 로 패킷 이벤트를 기록하는 것이었다(qhct.go, 부록 A). 그러나 v0.61 에서 API 가 크게 바뀌어 있었다 — 문서에 흔한 logging 패키지 기반 Tracer 는 제거되었고, 새 qlogwriter.Trace 경로를 붙였더니 파일이 0 바이트로 남은 채 이벤트가 기록되지 않고 프로세스가 블록되는 문제가 재현됐다. 원인 규명에 시간을 더 쓰는 대신 계측 지점을 옮기기로 했다.

전환의 논리: 우리가 필요한 것은 QUIC 내부 이벤트 전부가 아니라 '데이터그램이 몇 개, 몇 바이트 오갔는가'다. 그것은 소켓 경계에서 셀 수 있다 — 프로토콜 라이브러리를 뜯지 않고도.

5.4 countConn — 소켓 래핑 계측

```
type countConn struct {
    net.PacketConn                      // 원본 소켓을 품는다(임베딩)
    mu   sync.Mutex
    sent []int                          // 송신 데이터그램 크기 목록
    recv []int                          // 수신 데이터그램 크기 목록
}
func (c *countConn) WriteTo(b []byte, a net.Addr) (int, error) {
    c.mu.Lock(); c.sent = append(c.sent, len(b)); c.mu.Unlock()
    return c.PacketConn.WriteTo(b, a)   // 기록 후 원본에 위임
}
```

net.PacketConn 인터페이스를 만족하는 껍데기로 원본 UDP 소켓을 감싸고, WriteTo/ReadFrom 을 가로채 길이만 기록한 뒤 원본에 넘긴다(데코레이터 패턴). quic-go 가 이 소켓을 쓰게 만드는 열쇠가 Transport 주입이다:

```
tr := &quic.Transport{Conn: &countConn{PacketConn: udpConn}}
conn, err := tr.Dial(ctx, raddr, tlsConf, qconf)
```

quic.DialAddr()를 쓰면 라이브러리가 소켓을 내부에서 만들어 손댈 수 없다. Transport{Conn: …} 형태로 우리 소켓을 넣어야 모든 송수신이 래퍼를 통과한다. 특별 권한도, 외부 도구도, 라이브러리 수정도 필요 없다 — tcpdump(권한 문제)와 qlog(API 문제)를 모두 우회한 해법이며, 이후 증폭 실험 전체가 이 위에서 이뤄졌다.

실행 시 'connection doesn't allow setting of receive buffer size' 경고가 뜨는데, 이는 quic-go 가 래퍼를 *net.UDPConn 으로 캐스팅해 버퍼를 키우려다 실패했다는 알림이다. 수신 버퍼가 기본값으로 남을 뿐 측정 정확성에는 영향이 없어 무시했다.

5.5 패킷 실측과 무손실 기준선

키 교환	클라이언트 송신	서버 송신	무손실 HCT p50
X25519	2 개 / 2,560 B	1 개 / 1,280 B	3.704 ms
X25519MLKEM768	3 개 / 3,840 B	2 개 / 1,970 B	4.883 ms

계획서의 이론 예측(패킷 수 증가)이 실측으로 확인됐다. 무손실 고정 페널티는 +1.18 ms — TLS 의 +0.6 ms 보다 2 배 크다. QUIC 은 데이터그램을 1,280 B 단위로 자르고 첫 비행 안에서 직렬화 비용이 더해지기 때문으로 해석된다. 클라이언트 첫 비행은 대부분 일정했으나 반복 측정에서 PQC 쪽이 약 86% 3 개 / 14% 2 개로 변동했다 — 이 변동이 제 6 부 전이 꼬리 논의에서 다시 등장한다.

5.6 손실 스윕 — 가설 H1 의 기각

계획서 가설 H1: "PQC 는 패킷이 많아 손실 환경에서 신뢰성이 더 나빠진다." 손실 0.5·1·2·5% × 균일·버스트, 조건당 n=2,000 으로 검증했다.

지표	X25519	X25519MLKEM768	판정
PTO 발생률	12.0%	10.4%	z = 0.79 — 유의차 없음
백오프 2 단 도달	10 건	10 건	동일
백오프 3 단 도달	5 건	5 건	동일

패킷 수가 실제로 늘었음을 계측으로 확인한 상태에서도, 손실 조건의 신뢰성 저하는 관측되지 않았다 → H1 기각. QUIC 의 손실 감지·재전송이 패킷 수 증가를 상쇄한 것으로 해석된다. Montenegro 외(2025)의 'QUIC 이 KEM 간 격차를 완화한다'는 보고와 독립적으로 부합한다.

측정 방법: 각 연결의 시간열에서 PTO 발생 여부와 연속 만료 횟수(백오프 깊이)를 추출하는 분석 스크립트를 돌려 /tmp/det_*.txt 로 남겼다. H1 이 기각되면서 '적응형 FEC 로 손실을 복구한다'는 계획서 3 단계는 개선 대상을 잃었고 — 여기서 제 6 부의 질문이 시작된다.

제 6 부

증폭 제한의 발견 — 임계, 역전, 그리고 인과

6.1 동기 — 기각 이후의 질문

H1 이 기각된 뒤 남은 질문은 이것이었다: 손실이 문제가 아니라면, QUIC 에서 크기가 문제 되는 지점은 어디인가. RFC 9000 을 다시 읽으며 §8.1(1.9)에 주목했다 — 서버 송신 예산이 클라이언트 송신량의 3 배로 묶인다는 조문. 여기서 가설이 산술적으로 도출된다:

```
예산 = 클라이언트 첫 비행 × 3
X25519      : 2,560 B × 3 =  7,680 B
하이브리드   : 3,840 B × 3 = 11,520 B     ← 예산이 1.5 배
→ 서버 응답(≈인증서)이 7,680~11,520 B 사이라면, 고전만 추가 왕복을 겪는다
```

6.2 실험 도구 qamp — 인증서 크기라는 손잡이

검증하려면 서버 응답 크기를 마음대로 바꿔야 한다. qpkt.go 를 복사해 두 가지 패치를 붙였다(패치 스크립트 전문은 부록 A). 355 줄을 손으로 붙여넣는 대신 파이썬 문자열 치환으로 기존 파일을 변형하는 방식을 썼다 — 전송 오류를 줄이고 변경점을 명시적으로 남기기 위해서다.

패치 1 — makeCert(target): X.509 인증서의 ExtraExtensions 에 사설 OID(1.3.6.1.4.1.99999.1)로 더미 바이트 덩어리를 넣어 DER 인코딩 결과가 목표 크기가 되도록 부풀린다. 1 차 생성 후 실제 크기와 목표의 차이를 재보정하는 2 회 생성 방식으로 ±1 B 정확도를 얻었다(예: 목표 4,032 → 실제 4,033 B). 실제 ML-DSA 서명 없이 전송 바이트 수만 재현하는 접근으로, Kempf 외(2024)의 inflated chain 과 같다.

패치 2 — InitialPacketSize: quic-go Config 에서 찾아낸 필드(go doc 으로 발견, 하한 1,200 · 상한은 경로 MTU−28=1,472)로 클라이언트 Initial 데이터그램 크기를 조절하는 -initpkt 플래그를 추가했다. 6.8 의 인과 실험 손잡이다.

6.3 첫 시험의 실패 — RTT=0 이라는 함정

```
for cs in 500 4032 16000: 로컬(127.0.0.1)에서 certsize 확인
→ 인증서 501 / 4,033 / 16,000 B  (크기 조절 정확)
→ 그러나 HCT 는 셋 다 3~5 ms — 32 배 키웠는데 시간이 안 늘었다
```

이유는 단순했다 — 로컬 루프백은 RTT 가 사실상 0 이라, 추가 왕복이 생겨도 0.1 ms 도 안 되어 보이지 않는다. RTT 를 걸어야 왕복이 시간으로 나타난다. 흥미롭게도 선행연구 「A Quantum of QUIC」(2024)이 정확히 이 상태(RTT<0.1 ms 명시)에서 측정을 끝냈고, 그래서 증폭 비용을 볼 수 없었다(제 7 부). 우리는 같은 함정에 빠졌다가 빠져나온 셈이다.

6.4 RTT 50 ms — 갈림의 첫 관측

인증서	X25519 p50	하이브리드 p50	해석
500 B	54.1 ms	55.3 ms	둘 다 1-RTT
4,032 B (웹 중앙값)	104.8 ms	55.5 ms	★ 고전만 2-RTT — 49 ms 역전
16,000 B	106.6 ms	106.8 ms	둘 다 2-RTT

판정 기준은 이산적이다 — 왕복 50 ms 환경에서 1-RTT 면 약 54 ms, 2-RTT 면 약 105 ms 로 두 값만 나타난다. 같은 회차의 증폭비(수신/송신 총합)는 16,000 B 조건에서 X25519 6.21, 하이브리드 4.57 로 고전 쪽이 훨씬 높았다 — 분모(클라이언트 송신)가 작기 때문이며, 예산 압박이 고전 쪽에 먼저 온다는 정황이다(총합 지표의 한계는 6.9).

6.5 임계의 정밀 측정

개괄 스윕(500~16,000 B, 14 점)으로 전환 구간을 좁힌 뒤 100 B 간격 정밀 스윕(20 점 × 2 KEM, n=30)으로 임계를 특정했다. 원시 출력 전체는 부록 B.

키 교환	1→2 RTT 임계	임계 직전 서버 총수신	임계 직후

X25519	3,300 ~ 3,400 B	3,901 B	4,038 B
X25519MLKEM768	5,900 ~ 6,000 B	7,723 B	7,863 B

전환은 정확히 한 번, 계단형으로 일어났고(54→105 ms, 중간값 없음) 임계 전후 구간은 ±1 ms 이내로 평평했다. 임계에서의 서버 수신량이 각 예산 계산값(3,840 / 7,680 B) 부근에 위치한다 — 다만 이 수치는 핸드셰이크 전체 합산이므로 대조는 근사적이다(6.9). 두 임계 사이 약 3,400~5,900 B 가 후양자가 더 빠른 역전 구간이며, 폭은 약 2,500 B 다.

6.6 격차는 정확히 왕복 1 회 — RTT 비례성

RTT	X25519 p50	하이브리드 p50	격차	격차/RTT
10 ms	24.6	15.1	9.5	0.95
50 ms	104.8	55.6	49.2	0.98
100 ms	205.1	105.8	99.3	0.99
200 ms	403.2	205.4	197.8	0.99

```
하이브리드  HCT ≈ 1 × RTT + 5 ms      (1-RTT)
X25519     HCT ≈ 2 × RTT + 5 ms      (2-RTT)
```

인증서 4,032 B 고정, RTT 만 4 수준. 격차가 RTT 의 98~99%로 비례한다 — 차이의 정체가 '왕복 정확히 1 회'라는 결정적 증거다. 상수항 5 ms 는 처리·직렬화 비용이다. RTT 가 클수록(위성·국제 회선·모바일) 후양자의 이득이 커진다는, 직관과 정반대의 함의가 나온다.

6.7 재현성 — 이상치 하나와 전이 꼬리 하나

Tranco 실측 3 점(2,308/4,032/5,609 B)에서 n=200 으로 2 회 독립 실행을 했다. 다섯 조건은 완전히 일치했으나 5,609 B·하이브리드가 갈렸다 — 1 차 p50=105.7 ms(2-RTT), 2 차 p50=55.4 ms(1-RTT). 1 차의 min=54.9 ms 는 한 실행 안에 두 상태가 섞였음을 뜻한다.

검증을 위해 5,400~5,800 B 4 점을 각 n=100 으로 3 회 독립 반복했다. 결과: 5,609 B 이상은 3 회 모두 2-RTT 0% — 초기 관측은 300 회에서 한 번도 재현되지 않아 이상치로 처리했다. 유력 원인은 조건 전환 시 서버 프로세스 잔존(동일 포트 재사용 구조)이나 미확정이다. 이 사건이 2.4 의 '조건마다 다른 포트' 규율을 낳았다.

한편 5,400 B 에서는 3 회 모두 정확히 1%(100 회 중 1 건)의 2-RTT 가 나왔다 — 임계 아래에 폭 약 500 B 의 낮은 확률 전이 꼬리가 실재한다. 추정 원인은 클라이언트 첫 비행의 2~3 개 변동(86%/14%)이지만 관측된 1%와 14%가 일치하지 않아 단순 대응은 아니며, 원인 미규명으로 남긴다.

6.8 패딩 인과 실험 — 원인은 바이트 수 그 자체

여기까지는 상관관계다 — 키 교환을 바꾸면 키셰어·패킷 수·예산이 함께 변하므로, '후양자라서'인지 '많이 보내서'인지 분리되지 않는다. 그래서 KEM 을 X25519 로 고정하고 -initpkt 로 패딩만 조작했다. 예비 확인에서 송신 바이트가 설정을 정확히 따라감을 봤다(2,400/2,560/2,700/2,914 B, 패킷 수는 2 개 유지).

InitialPacketSize	첫 비행	예산(×3)	관측 임계
1,200 B	2,400 B	7,200 B	3,300 ~ 3,400 B
1,280 B (기본)	2,560 B	7,680 B	3,300 ~ 3,400 B
1,350 B	2,700 B	8,100 B	4,400 ~ 4,600 B
1,450 B	2,914 B	8,742 B	4,400 ~ 4,600 B

같은 알고리즘, 같은 인증서에서 패딩만으로 임계가 약 1,150 B 이동했다. 임계 이동의 원인은 키 교환 알고리즘이 아니라 클라이언트 송신 바이트 수 그 자체다 — 바이트의 내용은 무관하다. 이것이 '패딩을 FEC 복구 심볼로 대체한다'는 재정의의 실증적 토대다.

교환비: 데이터그램당 150 B 추가(1,200→1,350, 2 패킷 기준 300 B)로 감당 가능한 인증서가 약 1,150 B 늘었다 — 약 1:3.8 로, 3 배 증폭 규칙과 방향이 맞는다.

계단 구조: 예산 7,200 과 7,680 이 같은 임계, 8,100 과 8,742 가 같은 임계로 두 개씩 묶였다. 서버 응답이 1,280 B 데이터그램 단위로 나가므로 예산이 '데이터그램 하나 더'를 허용하는 경계에서만 임계가 움직인다는 추정과 부합하나 (7,200/1,280=5.6 개 → 8,100/1,280=6.3 개), 7,680(정확히 6.0)이 5 개 쪽에 묶인 이유는 헤더·AEAD 오버헤드로 짐작될 뿐 정확한 산식은 미규명이다. 논문에는 '관측된 계단 구조'로만 기술한다.

6.9 증폭비 총합이 3.0 을 넘는 이유

측정된 수신/송신 총합 비는 최대 6.21 로 한도 3.0 을 훌쩍 넘는다. 규정 위반이 아니다 — §8.1 의 제한은 주소 검증 전 구간에만 적용되고, 클라이언트 ACK 가 도착해 주소가 검증되면 해제된다. 우리 계측은 핸드셰이크 전체를 합산하므로 검증 전후가 섞인 값이다. 역으로 이 때문에 '검증 전 구간만의 예산 소진'을 직접 계측하지 못하며, 6.5 의 이론 대조가 근사에 머문다 — qlog 수준의 구간별 계측이 향후 과제다(제 9 부 한계 3).

6.10 함의 — 발견이 놓이는 자리

백분위 (Tranco 75k)	체인 크기	X25519	하이브리드
p5	2,308 B	1-RTT	1-RTT
p50 중앙값	4,032 B	2-RTT	1-RTT ★
p95	5,609 B	2-RTT	1-RTT ★

현재 웹 인증서의 중앙값부터 95 백분위까지가 역전 구간 안이다. 이득의 크기는 RTT 자체(50~200 ms)로, 무손실 페널티 1.18 ms 의 42~168 배다. 단, 이 이득은 전환기적이다 — 후양자 인증서(ML-DSA 체인 10~17 KB)가 배치되면 양쪽 다 2-RTT 가 되고, 인증서 압축(Abridged 등)이 보급되면 양쪽 다 1-RTT 가 되어 사라진다. 후양자 키 교환은 이미 배치되었으나 후양자 인증서는 아직인 지금이 정확히 역전 구간이 유효한 시기라는 점이, 한계이자 시의성이다.

제 7 부

선행연구 9 편 대조

비교 축은 다섯 가지다: ① 인증서 크기를 연속 변수로 훑어 임계를 탐색했는가 ② 정량적 임계 바이트를 제시했는가 ③ PQC 가 고전보다 빠른 구간을 보고했는가 ④ 격차의 RTT 비례를 정밀 검증했는가 ⑤ 실측 인증서 분포와 임계를 결합했는가.

검토 방법에 관한 정직한 단서: 9 편 중 상당수는 원문 전체가 아니라 타 AI 의 발췌·분석을 근거로 판정했다. 인용 문장이 구체적이어서 신빙성은 높다고 판단하나, 투고 전 원문 재확인 — 특히 「A Quantum of QUIC」의 RTT 서술과 Kampanakis 외의 Fig.4 — 은 필수 절차로 남는다.

논문	대상	①	②	③	④	⑤
본 연구	QUIC	O	O	O	O	O
Sakk 외 (Future Internet 2026)	IoT 해석모델	X	X	X	X	X
Nawrocki 외 (CoNEXT 2022)	QUIC 실측	X	X	X	X	부분
Fastly (McManus)	QUIC/압축	X	X	X	X	부분
ReACKed QUICer (IMC 2024)	QUIC	부분	X	X	부분	부분
A Quantum of QUIC (2024)	QUIC+PQC	X	X	X	X	X
Kampanakis & Kallitsis (2022)	TLS/QUIC	X	X	X	X	부분
Kampanakis & Weibel (2025)	TLS	부분	X	X	X	부분
Montenegro 외 (2025, TLS/QUIC 2 편)	TLS·QUIC+PQC	X	X	부분	부분	X
Chou & Cao (IEEE CNS 2026)	TCP/TLS+PQC	부분	O	X	부분	부분

7.1 논문별 요지와 본 연구와의 관계

Sakk 외 (2026) — $P=p^N$ 해석 모델로 '패킷 수 증가 → 신뢰성 저하'를 예측. 재전송을 무시한 모델임을 저자 스스로 인정. 본 연구의 H1 기각은 이 모델이 QUIC 실측에서 성립하지 않음을 보인 셈이다. 1 단계 수치 교차검증의 상대이기도 하다(제 3 부).

Nawrocki 외 (2022) — Tranco 1M 실측에서 클라이언트 초기 1,352 B 기준 61%가 증폭 한도 초과. 증폭 문제의 실재를 확립했고 클라이언트 크기를 변수로 다뤘으나 PQC 키 교환은 미포함.

ReACKed QUICer (2024) — 인증서 2 점 고정(1,212/5,113 B)으로 한도 이내/초과만 비교, Instant ACK 효과 중심. PQC 미다룸.

A Quantum of QUIC (2024) — quiche·LSQUIC 에서 KEM 별 TTFB 측정. 그러나 'RTT<0.1 ms 라 왕복 수는 무시 가능'이라 명시한 환경 — 증폭이 유발하는 시간 비용을 구조적으로 볼 수 없었고, Kyber 를 X25519 보다 항상 느린 것으로 보고했다. 본 연구가 6.3 에서 겪은 함정과 동일하며, 우리 기여의 핵심 방어선이다.

Kampanakis 계열 (2022·2025) — PQ 서명 체인이 증폭 윈도를 초과해 왕복을 유발함을 정성적으로 지적·시연. 서버 측(인증서) 관점이며 키 교환에 따른 임계 이동은 없음.

Montenegro 외 (2025, QUIC 판) — 손실 하에서 QUIC 이 KEM 간 격차를 완화(우리 H1 기각과 부합). 단 '하이브리드가 가장 느리다'고 보고 — 본 연구(하이브리드가 유리)와 표면상 반대다. 모순이 아니라 조건 차이다: 그쪽은 증폭 제한이 발동하지 않는 구성에서 순수 크기 비용만 봤고, 우리는 발동 구간을 봤다. 논문에서 반드시 설명해야 할 지점.

Chou & Cao (2026) — 방법론 최근접: 인증서 인위 팽창 + RTT 0/10/50/100/200 동일. 그러나 대상이 TCP/TLS 이고 임계는 초기 혼잡 윈도우(10/40 KB). TCP 윈도우는 클라이언트 송신량과 무관한 고정값이라 키 교환이 임계를 움직일 수 없다 — 역전 현상이 QUIC 고유임을 대조적으로 보여 주는 논문. QUIC 은 전혀 다루지 않음을 확인했다.

7.2 판정

③(PQC 역전 구간)과 ④(RTT 비례 정밀 검증)는 9 편 모두에서 보고되지 않았다 — 본 연구의 기여로 성립한다. ②는 Chou & Cao 가 제시했으나 TCP 윈도우 기준이며, QUIC 증폭 예산 기준의 임계(그리고 그 임계가 키 교환으로 이동한다는 사실)는 본 연구가 처음 정량화했다. 남은 위험은 신규성이 아니라 재현성(단일 구현)이다 — 제 9 부.

제 8 부

명령어 사전

본 연구에서 실제 사용한 명령을 범주별로 정리한다. 각 행: 명령(대표 형태) / 의미 / 본 연구에서의 용도.

8.1 셸 · 파일 · 텍스트 처리

명령	의미	용도
conda activate oreo_34	가상환경 활성화 — PATH·라이브러리 경로를 환경 내부로 교체	모든 세션의 첫 명령. 생략 시 go·openssl 3.5 를 못 찾음
cat > f << 'EOF' ... EOF	히어독 — EOF 까지의 입력을 파일로 기록. 구분자를 따옴표로 감싸면 $·백틱을 확장하지 않고 그대로 저장	코드 파일 전송의 표준 수단
cp / mv / rm -rf	복사 / 이동 / 삭제(-r 재귀, -f 확인 생략)	실행 파일 /tmp 복사 관례
ls -la	목록 상세(-l)·숨김 포함(-a)	산출물·권한 확인
find . -name X -type f -perm -u+x	이름·유형(파일)·실행권한으로 검색	quiche 바이너리 수색
wc -l	줄 수 세기	CSV 행 수 검증
head -N / tail -N / tail -f	앞·뒤 N 줄 / -f 는 파일 끝 실시간 추적	로그 감시
tee 파일	표준출력을 화면과 파일에 동시 기록	빌드 로그 보존
grep -E -v -c -A -B -n	패턴 검색: 확장정규식/반전/개수/뒤·앞 문맥/줄번호	결과 필터링 전반
sed -i 's/\r$//' f	줄 끝 CR 제거(-i 제자리 수정)	윈도 복붙로 생긴 CRLF 정리
sed -n 's/패턴\(캡처\).*/\1/p'	-n 자동출력 억제, \(..\) 캡처, /p 매칭 줄만 출력	측정기 출력에서 수치 추출
awk -F',' '{ ... }'	필드 구분자 지정 후 행별 계산	p50 집계·조건 분기
sort -n | uniq -c	수치 정렬 후 중복 개수 세기	분포 집계(2-RTT 비율 등)
printf "..." 값	서식 문자열 출력	진행 표시·CSV 조립

8.2 프로세스 제어

명령	의미	용도
CMD & / SRV=$!	백그라운드 실행 / $!는 직전 백그라운드 PID	서버 기동 후 PID 보관
kill $SRV (pkill -f 지양)	PID 지정 종료. pkill -f 는 패턴이 부모 셸까지 잡아 죽인 사고가 있어 금지 관례	조건 간 서버 정리
nohup CMD > log 2>&1 &	로그아웃 무관 지속, 표준출력·오류(2>&1)를 로그로	밤샘 스윕
timeout N CMD	N 초 후 강제 종료	모든 신규 실행의 필수 접두어
sleep N / wait PID	대기 / 자식 종료 대기	서버 기동 대기·정리
taskset -c 0-3 CMD	CPU 친화도 고정(코어 0~3)	측정 분산 억제
ps aux | grep 이름	프로세스 목록 확인	잔존 서버 점검

8.3 네임스페이스 · 네트워크

명령	의미	용도
unshare -Urnm bash -c '...'	사용자(-U)·네트워크(-n)·마운트(-m) 네임스페이스 생성 + 현재 사용자를 내부 root 로 매핑(-r)	sudo 없는 root 네트워크 작업
mount -t tmpfs none /run	/run 을 메모리 FS 로 덮어 쓰기 가능하게	ip netns 의 마운트 지점 요구
ip netns add P / ip netns exec P CMD	네임스페이스 생성 / 그 안에서 실행	peer(서버측) 분리
ip link add v0 type veth peer name v1	가상 이더넷 쌍 생성	격리 링크
ip link set v1 netns peer	인터페이스를 다른 ns 로 이동	loopback 우회 차단
ip addr add A/24 dev I · set mtu · set up	주소 부여·MTU 설정·활성화	링크 구성
ip route get IP	해당 목적지의 실제 경유 장치 조회	시도 2 의 lo 우회 진단
tc qdisc add dev I root netem SPEC / del	큐 규칙에 netem 부착 / 제거	조건 주입·초기화
netem delay Nms / loss P% / loss	고정 지연 / 균일 손실 / GE 버스트(1.11)	독립변수 구현

gemodel P R		
ping -c N -i S -W T 주소	N 회, S 초 간격, T 초 응답 대기	RTT 적용 검증(50.068 ms)
cat /sys/class/net/I/mtu	커널이 보는 실제 MTU	시도 1 의 미적용 진단

8.4 OpenSSL · Go · Rust

명령	의미	용도
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -subj /CN=localhost	자체서명(-x509) EC P-256 인증서 생성, 키 비암호화(-nodes), 대화 생략(-subj)	테스트 인증서 c.pem/k.pem
openssl s_server -tls1_3 -groups G -accept A -quiet	테스트 TLS 서버: 버전 고정·그룹 강제·조용히	TLS 스윕 상대
openssl s_client -connect A -groups G -msg -brief	테스트 클라이언트, -msg 핸드셰이크 메시지 덤프	키셰어 크기 확인
go mod init / go mod tidy	모듈 초기화 / 의존성 정리·다운로드	quic-go 도입
go doc 패키지.타입	API 문서 조회	InitialPacketSize 발견 경로
CGO_ENABLED=0 go build -o BIN f.go	C 연동 비활성 빌드	conda CC 함정 회피(5.2)
curl ... sh.rustup.rs | sh -s -- -y	rustup 공식 설치기(홈 디렉터리 설치, sudo 불요)	Rust 1.97.1 확보
cargo build --release -p PKG	워크스페이스에서 특정 패키지만 릴리스 빌드	quiche 시도(제 9 부)

제 9 부

함정 · 한계 · 향후 과제

9.1 함정 여덟 가지 — 겪은 순서대로

증상	원인	대처
go: command not found	conda 미활성화	conda activate oreo_34
cgo: C compiler not found	conda 가 CC 를 미설치 컴파일러로 지정	CGO_ENABLED=0 go build
MTU 변경이 적용 안 됨 (exit 0)	loopback 은 조용히 무시	veth 사용 + /sys 로 검증
측정이 lo 로 우회	veth 양끝이 같은 ns	중첩 ns 로 양끝 분리
Terminated 만 출력되고 셸 종료	pkill -f 패턴이 부모 셸 포함	PID 변수 보관 후 kill
경로를 찾지 못함	폴더명 '1311 신레오'의 공백	실행 파일 /tmp 복사
명령 무응답·무한 대기	타임아웃 부재	timeout N 필수 + 첫 실행은 n 작게
순수 ML-KEM 협상 실패	OpenSSL 3.5 서버 기본 그룹이 하이브리드 중심	양쪽 -groups 명시 + 협상 검증

아홉 번째로 기록할 만한 것: bash 예약변수 GROUPS 에 값을 넣어 조용히 덮인 사고 — 변수명을 KEMS 로 바꿔 해결했다. 예약어 목록은 반드시 피해서 명명한다.

9.2 방법론적 한계 일곱 가지

#	한계	영향과 완화
1	QUIC 단일 구현(quic-go v0.61.0)	구현별 패딩·코얼레싱 정책 차이로 임계값의 일반성 미검증. quiche 교차검증을 시도했으나 빌드 산출물이 생성되지 않아 미완(하단 기록) — 논문 한계로 명시
2	QUIC 은 하이브리드만 측정	Go crypto/tls 제약. 순수 ML-KEM-768(ek 1,184 B)은 하이브리드와 32 B 차라 유사 거동 예상 — 추정이며 미검증
3	증폭 예산의 구간 분리 불가	전체 합산 계측이라 검증 전 구간만 못 봄 → 이론 대조가 근사(6.9). qlog 수준 계측이 후속 과제
4	netem 시드 고정 불가	손실 패턴이 실행마다 상이 — 독립 실행 반복으로 완화
5	버스트 파라미터 임의성	gemodel 5% 50%에 문헌 근거 없음
6	단일 인증서로 체인 대체	전송 바이트는 등가이나 파싱 비용 차이 가능
7	시점 간 반복 부족	대부분 단일 시점 실행. 증폭 임계만 2~3 회 독립 반복 수행

9.3 quiche 교차검증 시도의 기록

Rust 1.31(conda) → 매니페스트 파싱 실패 → rustup 으로 1.97.1 설치 → 워크스페이스 빌드 'Finished' 표시. 그러나 target/release/에 최종 바이너리가 생성되지 않았다(build·deps·examples 디렉터리만 존재). -p quiche_apps 지정 빌드에서 quiche-client/server 가 한 차례 목격되었으나 이후 재확인에서 부재 — 원인 미규명(경로 공백 관여 의심). 약 1 시간 소요 후 '필수 아님' 판단으로 중단하고 한계 1 로 전환했다. 공백 없는 경로(/tmp) 재빌드가 다음 시도의 첫 수순이다.

9.4 향후 과제

과제	담당	내용
패딩-FEC 구현	황요한	Initial 패딩을 XOR 복구 심볼로 대체. 단계 0~4 와 A/B/C 방식은 FEC_인수인계.docx
다중 구현 교차검증	신레오	quiche 또는 ngtcp2 에서 임계 재현 — 논문 강도를 좌우
손실 × 인증서 2 차원 스윕	공동	지금까지 분리했던 두 축의 결합 — 실배치 조건 근사
검증 전 구간 계측	공동	qlog 또는 검증 시점 표시로 이론 대조 정밀화

부록 A

소스 코드 전문

본 연구에서 작성한 모든 코드의 전문이다. 총 7 개 파일 947 줄. 각 파일의 설계 의도는 해당 본문 부에서 설명했으며, 여기서는 실행 가능한 원본을 그대로 싣는다.

A.1 hct_v2.c — TLS 핸드셰이크 측정기 (120 줄)

2 차 TLS 스윕에 쓴 C 측정기. 논블로킹 connect + poll 타임아웃, 협상 그룹 검증, 백분위 산출을 포함한다. 4.2 절 참조.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <math.h>
#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <arpa/inet.h>
#include <openssl/ssl.h>
#include <openssl/err.h>
#include <openssl/objects.h>

static double now_ms(void){struct timespec ts;clock_gettime(CLOCK_MONOTONIC,&ts);
    return ts.tv_sec*1000.0+ts.tv_nsec/1.0e6;}
static int cmpd(const void*a,const void*b){double x=*(const double*)a,y=*(const double*)b;
    return (x>y)-(x<y);}
static double pct(double*s,int n,double p){if(n<=0)return 0.0;if(n==1)return s[0];
    double idx=(p/100.0)*(n-1);int lo=(int)idx,hi=lo+1<n?lo+1:n-1;double fr=idx-lo;
    return s[lo]*(1.0-fr)+s[hi]*fr;}

int main(int argc,char**argv){
  if(argc<6){fprintf(stderr,"usage: %s <host> <port> <group> <iters> <warmup> [csv] [timeout_ms]\n",argv[0]);return 2;}
  const char*host=argv[1];int port=atoi(argv[2]);const char*group=argv[3];
  int iters=atoi(argv[4]),warmup=atoi(argv[5]);const char*csv=(argc>6)?argv[6]:NULL;
  int TMO=(argc>7)?atoi(argv[7]):5000;

  SSL_CTX*ctx=SSL_CTX_new(TLS_client_method());
  if(!ctx){ERR_print_errors_fp(stderr);return 1;}
  SSL_CTX_set_min_proto_version(ctx,TLS1_3_VERSION);
  SSL_CTX_set_max_proto_version(ctx,TLS1_3_VERSION);
  SSL_CTX_set_verify(ctx,SSL_VERIFY_NONE,NULL);
  SSL_CTX_set_session_cache_mode(ctx,SSL_SESS_CACHE_OFF);
  SSL_CTX_set_options(ctx,SSL_OP_NO_TICKET);
  if(SSL_CTX_set1_groups_list(ctx,group)!=1){
    fprintf(stderr,"group '%s' not supported\n",group);
    ERR_print_errors_fp(stderr);SSL_CTX_free(ctx);return 1;}

  struct sockaddr_in addr;memset(&addr,0,sizeof addr);
  addr.sin_family=AF_INET;addr.sin_port=htons(port);
  if(inet_pton(AF_INET,host,&addr.sin_addr)!=1){
    fprintf(stderr,"bad addr %s\n",host);SSL_CTX_free(ctx);return 1;}

  int total=warmup+iters;
  double*all=calloc(iters,sizeof(double));
  FILE*fp=NULL;
  if(csv){fp=fopen(csv,"w");if(!fp){perror("fopen");return 1;}
    fprintf(fp,"group,iter,tcp_ms,tls_ms,total_ms,status,negotiated_group\n");}

  int ok=0,fail=0,tmo=0;const char*neg="?";
  for(int i=0;i<total;i++){
    int w=(i<warmup);double t0=now_ms();
    int fd=socket(AF_INET,SOCK_STREAM,0);
    if(fd<0){fail++;continue;}
    int one=1;setsockopt(fd,IPPROTO_TCP,TCP_NODELAY,&one,sizeof one);
    int fl=fcntl(fd,F_GETFL,0);fcntl(fd,F_SETFL,fl|O_NONBLOCK);

    int cr=connect(fd,(struct sockaddr*)&addr,sizeof addr);
    if(cr!=0&&errno!=EINPROGRESS){
      close(fd);fail++;
```

```
      if(fp&&!w)fprintf(fp,"%s,%d,,,,tcp_fail,\n",group,i-warmup);
      continue;}
    if(cr!=0){
      struct pollfd pfd={fd,POLLOUT,0};
      int pr=poll(&pfd,1,TMO);
      int soe=0;socklen_t sl=sizeof soe;
      if(pr<=0||getsockopt(fd,SOL_SOCKET,SO_ERROR,&soe,&sl)!=0||soe!=0){
        close(fd);fail++;if(pr==0)tmo++;
        if(fp&&!w)fprintf(fp,"%s,%d,,,,%s,\n",group,i-warmup,pr==0?"tcp_timeout":"tcp_fail");
        continue;}}
    double t1=now_ms();

    SSL*ssl=SSL_new(ctx);SSL_set_fd(ssl,fd);
    SSL_set_tlsext_host_name(ssl,"localhost");

    int r;double dl=t1+TMO;
    for(;;){
      r=SSL_connect(ssl);
      if(r==1)break;
      int e=SSL_get_error(ssl,r);
      if(e!=SSL_ERROR_WANT_READ&&e!=SSL_ERROR_WANT_WRITE){r=-1;break;}
      double rem=dl-now_ms();
      if(rem<=0){r=-2;break;}
      struct pollfd pfd={fd,(short)(e==SSL_ERROR_WANT_READ?POLLIN:POLLOUT),0};
      if(poll(&pfd,1,(int)rem)<=0){r=-2;break;}
    }
    double t2=now_ms();

    if(r==1){
      const char*n=SSL_group_to_name(ssl,SSL_get_negotiated_group(ssl));
      if(n)neg=n;
      if(!w){int k=i-warmup;all[k]=t2-t0;
        if(fp)fprintf(fp,"%s,%d,%.4f,%.4f,%.4f,ok,%s\n",group,k,t1-t0,t2-t1,t2-t0,n?n:"?");}
      ok++;
    }else{
      fail++;if(r==-2)tmo++;
      if(fp&&!w)fprintf(fp,"%s,%d,%.4f,%.4f,,%s,\n",group,i-warmup,t1-t0,t2-t1,
                        r==-2?"tls_timeout":"tls_fail");}
    SSL_free(ssl);close(fd);
  }
  if(fp)fclose(fp);

  int n=0;for(int i=0;i<iters;i++)if(all[i]>0.0)n++;
  double*s=calloc(n>0?n:1,sizeof(double));int j=0;double sum=0;
  for(int i=0;i<iters;i++)if(all[i]>0.0){s[j++]=all[i];sum+=all[i];}
  qsort(s,n,sizeof(double),cmpd);
  double mean=n?sum/n:0,var=0;
  for(int i=0;i<n;i++)var+=(s[i]-mean)*(s[i]-mean);
  double sd=n>1?sqrt(var/(n-1)):0;
  printf("group=%-18s negotiated=%-18s n=%d  fail=%d  tmo=%d  failrate=%.2f%%\n",
         group,neg,n,fail,tmo,(ok+fail)?100.0*fail/(ok+fail):0.0);
  if(n)printf("  HCT(ms)  mean=%.3f  sd=%.3f  p50=%.3f  p95=%.3f  p99=%.3f  min=%.3f  max=%.3f\n",
         mean,sd,pct(s,n,50),pct(s,n,95),pct(s,n,99),s[0],s[n-1]);
  free(s);free(all);SSL_CTX_free(ctx);return 0;
}
```

A.2 run_v3.sh — TLS 스윕 오케스트레이터 (106 줄)

주의: 아래 원본은 netem 을 v0 한쪽에만 거는 단방향 초판이다(delay=RTT). 최종 스윕과 이후 QUIC·증폭 실험에는 양방향 대칭(각 RTT/2) 패치본을 썼다. 2.3·4.4 절 참조.

```
#!/usr/bin/env bash
# ============================================================
#  확장 스윕 (B) — veth 링크 + MTU 1500 + 타임아웃
#
#  중첩 네트워크 네임스페이스로 veth 양끝을 분리한다.
#  loopback은 MTU 변경이 커널에서 무시되므로(exit=0이나 미적용)
#  실제 이더넷처럼 동작하는 veth 링크를 쓴다.
#
#    [기본 ns]  v0 10.9.0.1  <---- veth ---->  v1 10.9.0.2  [peer ns]
#       클라이언트(hct_v2)                        서버(s_server)
#
#  netem은 v0(송신측)에 걸어 단방향으로 적용 → delay = RTT (loopback과 달리 2배 아님)
# ============================================================
set -u
QUICK="${1:-full}"
PORT=45577
```

```
OUT="$(pwd)/resultsB_$(date +%Y%m%d_%H%M%S)"
BIN="$(pwd)/hct_v2"
CERT="$(pwd)/c.pem"; KEY="$(pwd)/k.pem"
TMO=5000

KEMS="X25519 MLKEM512 MLKEM768 MLKEM1024 X25519MLKEM768"
if [ "$QUICK" = "quick" ]; then
  RTTS="0 50"; LOSSES="0 2"; MODELS="uniform"; ITERS=100; WARMUP=10
else
  RTTS="0 10 50 100 200"; LOSSES="0 0.5 1 2 5"
  MODELS="uniform burst"; ITERS=2000; WARMUP=50
fi

[ -x "$BIN" ] || { echo "오류: hct_v2 없음. 먼저 컴파일하세요."; exit 1; }
[ -f "$CERT" ] || openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 \
  -keyout "$KEY" -out "$CERT" -days 30 -nodes -subj "/CN=localhost" 2>/dev/null
mkdir -p "$OUT"
echo "출력: $OUT"
export OUT BIN CERT KEY PORT KEMS RTTS LOSSES MODELS ITERS WARMUP TMO

unshare -Urnm bash <<'INNER'
set -u
# --- veth 링크 구성 ---
mount -t tmpfs none /run 2>/dev/null
mkdir -p /run/netns
ip netns add peer
ip link add v0 type veth peer name v1
ip link set v1 netns peer
ip addr add 10.9.0.1/24 dev v0
ip link set v0 mtu 1500
ip link set v0 up
ip netns exec peer sh -c "ip addr add 10.9.0.2/24 dev v1; ip link set v1 mtu 1500; ip link set v1 up; ip link set lo up"

echo "링크 구성: v0(10.9.0.1) <-> v1(10.9.0.2)"
echo "v0 MTU = $(cat /sys/class/net/v0/mtu)"
ping -c 1 -W 2 10.9.0.2 >/dev/null 2>&1 && echo "연결 확인 OK" || { echo "연결 실패 — 중단"; exit 1; }

SUM="$OUT/summary.csv"
echo "model,rtt_ms,loss_pct,group,n,fail,tmo,fail_rate_pct,mean_ms,sd_ms,p50_ms,p95_ms,p99_ms" > "$SUM"
tot=0
for m in $MODELS; do for r in $RTTS; do for l in $LOSSES; do for g in $KEMS; do tot=$((tot+1)); done;done;done;done
cnt=0
for model in $MODELS; do
 for rtt in $RTTS; do
  for loss in $LOSSES; do
    # netem을 v0 송신측에만 적용 → 단방향. delay 값 = 목표 RTT
    tc qdisc del dev v0 root 2>/dev/null
    spec=""
    [ "$rtt" != "0" ] && spec="delay ${rtt}ms"
    if [ "$loss" != "0" ]; then
      if [ "$model" = "burst" ]; then spec="$spec loss gemodel ${loss}% 50%"
      else spec="$spec loss ${loss}%"; fi
    fi
    [ -n "$spec" ] && tc qdisc add dev v0 root netem $spec

    for group in $KEMS; do
      cnt=$((cnt+1))
      it=$ITERS
      [ "$loss" = "0" ] && it=$((ITERS/8))
      case "$rtt" in 100|200) it=$((it/2));; esac
      printf "[%3d/%3d] %-7s RTT=%-4s loss=%-4s %-16s " "$cnt" "$tot" "$model" "$rtt" "$loss" "$group"

      # 서버는 peer ns 안에서 기동
      ip netns exec peer openssl s_server -cert "$CERT" -key "$KEY" -tls1_3 \
        -groups "$group" -accept 10.9.0.2:$PORT -quiet >/dev/null 2>&1 &
      SRV=$!; sleep 0.8

      csv="$OUT/raw_${model}_rtt${rtt}_loss${loss}_${group}.csv"
      out=$("$BIN" 10.9.0.2 $PORT "$group" "$it" "$WARMUP" "$csv" "$TMO" 2>/dev/null)
      neg=$(echo "$out"|sed -n 's/.*negotiated=\([A-Za-z0-9_]*\).*/\1/p')
      n=$(echo  "$out"|head -1|sed -n 's/.*[[:space:]]n=\([0-9]*\)[[:space:]].*/\1/p')
      f=$(echo  "$out"|sed -n 's/.*fail=\([0-9]*\).*/\1/p')
      tm=$(echo "$out"|sed -n 's/.*tmo=\([0-9]*\).*/\1/p')
      fr=$(echo "$out"|sed -n 's/.*failrate=\([0-9.]*\)%.*/\1/p')
      mm=$(echo "$out"|sed -n 's/.*mean=\([0-9.]*\).*/\1/p')
      sd=$(echo "$out"|sed -n 's/.*sd=\([0-9.]*\).*/\1/p')
      p50=$(echo "$out"|sed -n 's/.*p50=\([0-9.]*\).*/\1/p')
      p95=$(echo "$out"|sed -n 's/.*p95=\([0-9.]*\).*/\1/p')
      p99=$(echo "$out"|sed -n 's/.*p99=\([0-9.]*\).*/\1/p')
```

```
      [ "${neg^^}" != "${group^^}" ] && printf "!!협상불일치(%s) " "$neg"
      printf "p50=%-9s 실패=%-6s tmo=%s\n" "${p50:--}" "${fr:--}" "${tm:-0}"
      echo "$model,$rtt,$loss,$group,${n:-0},${f:-0},${tm:-0},${fr:-0},${mm:-},${sd:-},${p50:-},${p95:-},${p99:-}" >>
"$SUM"
      kill $SRV 2>/dev/null; wait $SRV 2>/dev/null
    done
  done
 done
done
tc qdisc del dev v0 root 2>/dev/null
echo ""; echo "완료: $SUM"
INNER
```

A.3 qhct.go — QUIC 핸드셰이크 측정기 (371 줄)

quic-go 기반 HCT 측정기. qlog 대신 countConn 래핑으로 데이터그램을 계측한다. 5.1~5.3 절 참조.

```
// qhct — QUIC 핸드셰이크 완료 시간(HCT) 측정 도구
//
// 서버·클라이언트 모드를 하나의 바이너리로 제공한다.
// qlog를 켜면 Initial 패킷 수·증폭 예산 동작을 패킷 단위로 관찰할 수 있다.
//
// 빌드:  go build -o qhct qhct.go
// 서버:  ./qhct -mode server -addr 10.9.0.2:44433 -group X25519MLKEM768
// 측정:  ./qhct -mode client -addr 10.9.0.2:44433 -group X25519MLKEM768 \
//              -iters 500 -warmup 20 -csv out.csv -timeout 5000
// qlog:  -qlog ./qlogdir  (클라이언트/서버 어느 쪽이든)
package main

import (
    "context"
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/tls"
    "crypto/x509"
    "crypto/x509/pkix"
    "encoding/csv"
    "flag"
    "fmt"
    "log"
    "math"
    "math/big"
    "net"
    "os"
    "path/filepath"
    "sort"
    "strconv"
    "time"

    "github.com/quic-go/quic-go"
    "github.com/quic-go/quic-go/logging"
    "github.com/quic-go/quic-go/qlog"
)

const alpn = "rne-hct"

// 자체 서명 인증서 생성 (측정 대상은 키 교환이므로 검증은 생략)
func genCert() tls.Certificate {
    key, err := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    if err != nil {
        log.Fatal(err)
    }
    tmpl := x509.Certificate{
        SerialNumber: big.NewInt(1),
        Subject:      pkix.Name{CommonName: "localhost"},
        NotBefore:    time.Now().Add(-time.Hour),
        NotAfter:     time.Now().Add(24 * 365 * time.Hour),
        DNSNames:     []string{"localhost"},
    }
    der, err := x509.CreateCertificate(rand.Reader, &tmpl, &tmpl, &key.PublicKey, key)
    if err != nil {
        log.Fatal(err)
    }
    return tls.Certificate{Certificate: [][]byte{der}, PrivateKey: key}
}

// 그룹 이름 → CurveID
func curveOf(name string) (tls.CurveID, error) {
    switch name {
    case "X25519":
```

```
        return tls.X25519, nil
    case "P256":
        return tls.CurveP256, nil
    case "P384":
        return tls.CurveP384, nil
    case "X25519MLKEM768":
        return tls.X25519MLKEM768, nil
    default:
        return 0, fmt.Errorf("지원하지 않는 그룹: %s (X25519 | P256 | P384 | X25519MLKEM768)", name)
    }
}

func pct(sorted []float64, p float64) float64 {
    n := len(sorted)
    if n == 0 {
        return 0
    }
    if n == 1 {
        return sorted[0]
    }
    idx := (p / 100.0) * float64(n-1)
    lo := int(idx)
    hi := lo + 1
    if hi >= n {
        hi = n - 1
    }
    fr := idx - float64(lo)
    return sorted[lo]*(1-fr) + sorted[hi]*fr
}

func qlogTracer(dir, tag string) func(context.Context, logging.Perspective, quic.ConnectionID) *logging.ConnectionTracer {
    if dir == "" {
        return nil
    }
    os.MkdirAll(dir, 0755)
    return func(_ context.Context, p logging.Perspective, cid quic.ConnectionID) *logging.ConnectionTracer {
        role := "client"
        if p == logging.PerspectiveServer {
            role = "server"
        }
        fn := filepath.Join(dir, fmt.Sprintf("%s_%s_%x.qlog", tag, role, cid.Bytes()))
        fh, err := os.Create(fn)
        if err != nil {
            return nil
        }
        return qlog.NewConnectionTracer(fh, p, cid)
    }
}

func runServer(addr, group, qlogDir string) {
    cid, err := curveOf(group)
    if err != nil {
        log.Fatal(err)
    }
    tlsConf := &tls.Config{
        Certificates:     []tls.Certificate{genCert()},
        NextProtos:       []string{alpn},
        MinVersion:       tls.VersionTLS13,
        CurvePreferences: []tls.CurveID{cid},
        SessionTicketsDisabled: true, // 세션 재개 차단 — 전체 핸드셰이크만 측정
    }
    qConf := &quic.Config{
        Tracer:               qlogTracer(qlogDir, "srv"),
        MaxIdleTimeout:       30 * time.Second,
        Allow0RTT:            false,
        DisablePathMTUDiscovery: true,
    }
    ln, err := quic.ListenAddr(addr, tlsConf, qConf)
    if err != nil {
        log.Fatal("리스닝 실패: ", err)
    }
    fmt.Printf("QUIC 서버 기동: %s  group=%s\n", addr, group)
    for {
        conn, err := ln.Accept(context.Background())
        if err != nil {
            continue
        }
        go func(c *quic.Conn) {
            // 핸드셰이크만 필요하므로 즉시 종료
            time.Sleep(5 * time.Millisecond)
            c.CloseWithError(0, "done")
        }(conn)
    }
}
```

```
func runClient(addr, group string, iters, warmup, timeoutMs int, csvPath, qlogDir string) {
    cid, err := curveOf(group)
    if err != nil {
        log.Fatal(err)
    }
    tlsConf := &tls.Config{
        InsecureSkipVerify: true, // 자체 서명. 측정 대상은 키 교환
        NextProtos:         []string{alpn},
        MinVersion:         tls.VersionTLS13,
        CurvePreferences:   []tls.CurveID{cid},
        ClientSessionCache: nil, // 세션 재개 차단
    }

    var w *csv.Writer
    if csvPath != "" {
        fh, err := os.Create(csvPath)
        if err != nil {
            log.Fatal(err)
        }
        defer fh.Close()
        w = csv.NewWriter(fh)
        defer w.Flush()
        w.Write([]string{"group", "iter", "hct_ms", "status", "negotiated_group", "err"})
    }

    total := warmup + iters
    var samples []float64
    okN, failN, tmoN := 0, 0, 0
    negName := "?"

    for i := 0; i < total; i++ {
        isWarm := i < warmup
        ctx, cancel := context.WithTimeout(context.Background(), time.Duration(timeoutMs)*time.Millisecond)

        qConf := &quic.Config{
            Tracer:                  qlogTracer(qlogDir, fmt.Sprintf("cli%d", i)),
            HandshakeIdleTimeout:    time.Duration(timeoutMs) * time.Millisecond,
            MaxIdleTimeout:          30 * time.Second,
            DisablePathMTUDiscovery: true,
        }

        t0 := time.Now()
        conn, err := quic.DialAddr(ctx, addr, tlsConf, qConf)
        el := float64(time.Since(t0).Microseconds()) / 1000.0
        cancel()

        if err != nil {
            failN++
            status := "fail"
            if ctx.Err() == context.DeadlineExceeded || err == context.DeadlineExceeded {
                tmoN++
                status = "timeout"
            }
            if w != nil && !isWarm {
                w.Write([]string{group, strconv.Itoa(i - warmup), "", status, "", err.Error()})
            }
            continue
        }

        st := conn.ConnectionState()
        if n := st.TLS.CurveID; n != 0 {
            negName = curveName(n)
        }
        conn.CloseWithError(0, "bye")

        okN++
        if !isWarm {
            samples = append(samples, el)
            if w != nil {
                w.Write([]string{group, strconv.Itoa(i - warmup),
                    fmt.Sprintf("%.4f", el), "ok", negName, ""})
            }
        }
    }

    sort.Float64s(samples)
    n := len(samples)
    mean, sd := 0.0, 0.0
    for _, v := range samples {
        mean += v
    }
    if n > 0 {
        mean /= float64(n)
        for _, v := range samples {
            sd += (v - mean) * (v - mean)
```

```
        }
        if n > 1 {
            sd = math.Sqrt(sd / float64(n-1))
        }
    }
    frate := 0.0
    if okN+failN > 0 {
        frate = 100.0 * float64(failN) / float64(okN+failN)
    }
    fmt.Printf("group=%-18s negotiated=%-18s n=%d  fail=%d  tmo=%d  failrate=%.2f%%\n",
        group, negName, n, failN, tmoN, frate)
    if n > 0 {
        fmt.Printf("  HCT(ms)  mean=%.3f  sd=%.3f  p50=%.3f  p95=%.3f  p99=%.3f  min=%.3f  max=%.3f\n",
            mean, sd, pct(samples, 50), pct(samples, 95), pct(samples, 99),
            samples[0], samples[n-1])
    }
}

func curveName(c tls.CurveID) string {
    switch c {
    case tls.X25519:
        return "X25519"
    case tls.CurveP256:
        return "P256"
    case tls.CurveP384:
        return "P384"
    case tls.X25519MLKEM768:
        return "X25519MLKEM768"
    }
    return fmt.Sprintf("id_%d", uint16(c))
}


// countingConn — UDP 데이터그램의 개수와 크기를 기록하는 래퍼.
// tcpdump 없이, 권한 없이 패킷 수준 관찰이 가능하다.
type countingConn struct {
    net.PacketConn
    sent []int // 송신 데이터그램 크기 (바이트)
    rcvd []int // 수신 데이터그램 크기
}

func (c *countingConn) WriteTo(b []byte, addr net.Addr) (int, error) {
    c.sent = append(c.sent, len(b))
    return c.PacketConn.WriteTo(b, addr)
}

func (c *countingConn) ReadFrom(b []byte) (int, net.Addr, error) {
    n, addr, err := c.PacketConn.ReadFrom(b)
    if err == nil {
        c.rcvd = append(c.rcvd, n)
    }
    return n, addr, err
}

func sum(xs []int) int {
    t := 0
    for _, x := range xs {
        t += x
    }
    return t
}

// 단일 핸드셰이크를 수행하며 데이터그램 크기를 보고한다
func runProbe(addr, group string, timeoutMs int) {
    cid, err := curveOf(group)
    if err != nil {
        log.Fatal(err)
    }
    udpAddr, err := net.ResolveUDPAddr("udp", addr)
    if err != nil {
        log.Fatal(err)
    }
    pc, err := net.ListenUDP("udp", &net.UDPAddr{IP: net.IPv4zero, Port: 0})
    if err != nil {
        log.Fatal(err)
    }
    cc := &countingConn{PacketConn: pc}
    tr := &quic.Transport{Conn: cc}
    defer tr.Close()

    tlsConf := &tls.Config{
        InsecureSkipVerify: true,
        NextProtos:         []string{alpn},
        MinVersion:         tls.VersionTLS13,
        CurvePreferences:   []tls.CurveID{cid},
```

```
    }
    qConf := &quic.Config{
        HandshakeIdleTimeout:    time.Duration(timeoutMs) * time.Millisecond,
        MaxIdleTimeout:          10 * time.Second,
        DisablePathMTUDiscovery: true,
    }
    ctx, cancel := context.WithTimeout(context.Background(), time.Duration(timeoutMs)*time.Millisecond)
    defer cancel()

    t0 := time.Now()
    conn, err := tr.Dial(ctx, udpAddr, tlsConf, qConf)
    el := float64(time.Since(t0).Microseconds()) / 1000.0
    if err != nil {
        fmt.Printf("group=%s  핸드셰이크 실패: %v\n", group, err)
        fmt.Printf("  송신 %d개 %v\n", len(cc.sent), cc.sent)
        return
    }
    neg := curveName(conn.ConnectionState().TLS.CurveID)
    conn.CloseWithError(0, "bye")
    time.Sleep(50 * time.Millisecond)

    fmt.Printf("group=%-18s negotiated=%-18s HCT=%.3fms\n", group, neg, el)
    fmt.Printf("  클라이언트 송신: %d개 데이터그램, 총 %d B  %v\n", len(cc.sent), sum(cc.sent), cc.sent)
    fmt.Printf("  클라이언트 수신: %d개 데이터그램, 총 %d B  %v\n", len(cc.rcvd), sum(cc.rcvd), cc.rcvd)
    if len(cc.sent) > 0 && len(cc.rcvd) > 0 {
        fmt.Printf("  증폭비(수신/송신): %.2f  (RFC 9000 §8.1 한도 3.0)\n",
            float64(sum(cc.rcvd))/float64(sum(cc.sent)))
    }
}

func main() {
    mode := flag.String("mode", "client", "server | client | probe")
    addr := flag.String("addr", "127.0.0.1:44433", "주소:포트")
    group := flag.String("group", "X25519MLKEM768", "키 교환 그룹")
    iters := flag.Int("iters", 100, "측정 반복 횟수")
    warmup := flag.Int("warmup", 10, "워밍업 횟수 (통계 제외)")
    timeout := flag.Int("timeout", 5000, "핸드셰이크 타임아웃 (ms)")
    csvPath := flag.String("csv", "", "원자료 CSV 경로")
    qlogDir := flag.String("qlog", "", "qlog 출력 디렉터리 (비우면 비활성)")
    flag.Parse()

    switch *mode {
    case "server":
        runServer(*addr, *group, *qlogDir)
    case "client":
        runClient(*addr, *group, *iters, *warmup, *timeout, *csvPath, *qlogDir)
    case "probe":
        runProbe(*addr, *group, *timeout)
    default:
        log.Fatal("mode는 server | client | probe")
    }
}
```

A.4 qpkt.go — QUIC 패킷 계측 도구 (232 줄)

방향별 데이터그램 수·크기를 기록하는 도구. 증폭 실험의 기반이 되었다. 5.4 절 참조.

```
// qpkt — QUIC 핸드셰이크 중 송신 데이터그램을 직접 계측한다.
//
// quic.Transport 에 래핑한 PacketConn 을 넘겨, WriteTo/ReadFrom 을 가로채
// 방향별 데이터그램 개수·크기를 기록한다. tcpdump 나 특별 권한이 필요 없다.
//
// 빌드: CGO_ENABLED=0 go build -o qpkt qpkt.go
// 서버: ./qpkt -mode server -addr 127.0.0.1:44433 -group X25519MLKEM768
// 측정: ./qpkt -mode client -addr 127.0.0.1:44433 -group X25519MLKEM768 -n 5
package main

import (
    "context"
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/tls"
    "crypto/x509"
    "crypto/x509/pkix"
    "flag"
    "fmt"
    "log"
    "math/big"
    "net"
```

```
    "sync"
    "time"

    "github.com/quic-go/quic-go"
)

const alpn = "rne-pkt"

// ---- 계측용 PacketConn 래퍼 ----
type countConn struct {
    net.PacketConn
    mu       sync.Mutex
    sent     []int // 송신 데이터그램 크기 (bytes)
    recv     []int // 수신 데이터그램 크기
    counting bool
}

func (c *countConn) WriteTo(b []byte, addr net.Addr) (int, error) {
    c.mu.Lock()
    if c.counting {
        c.sent = append(c.sent, len(b))
    }
    c.mu.Unlock()
    return c.PacketConn.WriteTo(b, addr)
}

func (c *countConn) ReadFrom(b []byte) (int, net.Addr, error) {
    n, a, err := c.PacketConn.ReadFrom(b)
    if err == nil {
        c.mu.Lock()
        if c.counting {
            c.recv = append(c.recv, n)
        }
        c.mu.Unlock()
    }
    return n, a, err
}

func (c *countConn) start() {
    c.mu.Lock()
    c.sent = nil
    c.recv = nil
    c.counting = true
    c.mu.Unlock()
}

func (c *countConn) stop() ([]int, []int) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.counting = false
    s := append([]int(nil), c.sent...)
    r := append([]int(nil), c.recv...)
    return s, r
}

func sum(a []int) int {
    t := 0
    for _, v := range a {
        t += v
    }
    return t
}

func genCert() tls.Certificate {
    key, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    tmpl := x509.Certificate{
        SerialNumber: big.NewInt(1),
        Subject:      pkix.Name{CommonName: "localhost"},
        NotBefore:    time.Now().Add(-time.Hour),
        NotAfter:     time.Now().Add(24 * 365 * time.Hour),
        DNSNames:     []string{"localhost"},
    }
    der, _ := x509.CreateCertificate(rand.Reader, &tmpl, &tmpl, &key.PublicKey, key)
    return tls.Certificate{Certificate: [][]byte{der}, PrivateKey: key}
}

func curveOf(name string) (tls.CurveID, error) {
    switch name {
    case "X25519":
        return tls.X25519, nil
    case "P256":
        return tls.CurveP256, nil
    case "P384":
        return tls.CurveP384, nil
    case "X25519MLKEM768":
```

```
        return tls.X25519MLKEM768, nil
    }
    return 0, fmt.Errorf("지원하지 않는 그룹: %s", name)
}

func runServer(addr, group string) {
    cid, err := curveOf(group)
    if err != nil {
        log.Fatal(err)
    }
    udpAddr, err := net.ResolveUDPAddr("udp", addr)
    if err != nil {
        log.Fatal(err)
    }
    pc, err := net.ListenUDP("udp", udpAddr)
    if err != nil {
        log.Fatal(err)
    }
    tr := &quic.Transport{Conn: pc}
    tlsConf := &tls.Config{
        Certificates:           []tls.Certificate{genCert()},
        NextProtos:             []string{alpn},
        MinVersion:             tls.VersionTLS13,
        CurvePreferences:       []tls.CurveID{cid},
        SessionTicketsDisabled: true,
    }
    ln, err := tr.Listen(tlsConf, &quic.Config{
        MaxIdleTimeout:          30 * time.Second,
        DisablePathMTUDiscovery: true,
    })
    if err != nil {
        log.Fatal(err)
    }
    fmt.Printf("서버 기동: %s  group=%s\n", addr, group)
    for {
        conn, err := ln.Accept(context.Background())
        if err != nil {
            continue
        }
        go func(c *quic.Conn) {
            time.Sleep(5 * time.Millisecond)
            c.CloseWithError(0, "done")
        }(conn)
    }
}

func runClient(addr, group string, n int) {
    cid, err := curveOf(group)
    if err != nil {
        log.Fatal(err)
    }
    raddr, err := net.ResolveUDPAddr("udp", addr)
    if err != nil {
        log.Fatal(err)
    }
    pc, err := net.ListenUDP("udp", &net.UDPAddr{IP: net.IPv4zero, Port: 0})
    if err != nil {
        log.Fatal(err)
    }
    cc := &countConn{PacketConn: pc}
    tr := &quic.Transport{Conn: cc}
    tlsConf := &tls.Config{
        InsecureSkipVerify: true,
        NextProtos:         []string{alpn},
        MinVersion:         tls.VersionTLS13,
        CurvePreferences:   []tls.CurveID{cid},
    }

    fmt.Printf("=== %s ===\n", group)
    var totSent, totRecv, cntS, cntR int
    for i := 0; i < n; i++ {
        ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
        cc.start()
        t0 := time.Now()
        conn, err := tr.Dial(ctx, raddr, tlsConf, &quic.Config{
            HandshakeIdleTimeout:    5 * time.Second,
            MaxIdleTimeout:          30 * time.Second,
            DisablePathMTUDiscovery: true,
        })
        el := float64(time.Since(t0).Microseconds()) / 1000.0
        s, r := cc.stop()
        cancel()
        if err != nil {
            fmt.Printf("  [%d] 실패: %v\n", i, err)
            continue
```

```
        }
        conn.CloseWithError(0, "bye")

        if i == 0 {
            fmt.Printf("  송신 데이터그램: %v  (합계 %dB)\n", s, sum(s))
            fmt.Printf("  수신 데이터그램: %v  (합계 %dB)\n", r, sum(r))
        }
        fmt.Printf("  [%d] HCT=%.3fms  송신 %d개/%dB  수신 %d개/%dB\n",
            i, el, len(s), sum(s), len(r), sum(r))
        totSent += sum(s)
        totRecv += sum(r)
        cntS += len(s)
        cntR += len(r)
    }
    if n > 0 {
        fmt.Printf("  평균: 송신 %.1f개 %.0fB · 수신 %.1f개 %.0fB\n",
            float64(cntS)/float64(n), float64(totSent)/float64(n),
            float64(cntR)/float64(n), float64(totRecv)/float64(n))
        fmt.Printf("  증폭비(수신/송신): %.2f  ← RFC 9000 §8.1 서버 한도 3배\n",
            float64(totRecv)/float64(totSent))
    }
}

func main() {
    mode := flag.String("mode", "client", "server 또는 client")
    addr := flag.String("addr", "127.0.0.1:44433", "주소:포트")
    group := flag.String("group", "X25519MLKEM768", "키 교환 그룹")
    n := flag.Int("n", 5, "반복 횟수")
    flag.Parse()

    switch *mode {
    case "server":
        runServer(*addr, *group)
    case "client":
        runClient(*addr, *group, *n)
    default:
        log.Fatal("mode는 server 또는 client")
    }
}
```

A.5 patch1_certsize.py — 인증서 크기 팽창 패치 (63 줄)

qpkt.go에 -certsize 옵션을 주입한다. X.509 확장에 더미 바이트를 삽입해 목표 크기로 부풀린다. 6.2 절 참조.

```
# qamp.go 생성 패치 1 — 인증서 크기 조절 기능 추가 (qpkt.go 복사본에 적용)
# 사용: cp qpkt.go qamp.go && python3 patch1_certsize.py
import re
p='qamp.go'; s=open(p).read()

# 1) asn1 임포트 추가
s=s.replace('\t"crypto/x509/pkix"\n', '\t"crypto/x509/pkix"\n\t"encoding/asn1"\n')

# 2) genCert -> makeCert(target) 로 교체
old=re.search(r'func genCert\(\).*?\n}\n', s, flags=re.S).group(0)
new='''func makeCert(target int) (tls.Certificate, int) {
    build := func(pad int) ([]byte, *ecdsa.PrivateKey) {
        key, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
        tmpl := x509.Certificate{
            SerialNumber: big.NewInt(1),
            Subject:      pkix.Name{CommonName: "localhost"},
            NotBefore:    time.Now().Add(-time.Hour),
            NotAfter:     time.Now().Add(24 * 365 * time.Hour),
            DNSNames:     []string{"localhost"},
        }
        if pad > 0 {
            blob := make([]byte, pad)
            rand.Read(blob)
            tmpl.ExtraExtensions = []pkix.Extension{{
                Id:    asn1.ObjectIdentifier{1, 3, 6, 1, 4, 1, 99999, 1},
                Value: blob,
            }}
        }
        der, err := x509.CreateCertificate(rand.Reader, &tmpl, &tmpl, &key.PublicKey, key)
        if err != nil {
            log.Fatal(err)
        }
        return der, key
    }
    base, _ := build(0)
```

```
    pad := 0
    if target > len(base) {
        pad = target - len(base)
    }
    der, key := build(pad)
    if diff := target - len(der); pad > 0 && diff != 0 && pad+diff > 0 {
        der, key = build(pad + diff)
    }
    return tls.Certificate{Certificate: [][]byte{der}, PrivateKey: key}, len(der)
}
'''
s=s.replace(old,new)

# 3) runServer 에 certSize 인자
s=s.replace('func runServer(addr, group string) {','func runServer(addr, group string, certSize int) {')
s=s.replace('Certificates:           []tls.Certificate{genCert()},',
            'Certificates:           []tls.Certificate{cert},')
s=s.replace('\tua, err := net.ResolveUDPAddr("udp", addr)',
            '\tcert, actualSize := makeCert(certSize)\n\tua, err := net.ResolveUDPAddr("udp", addr)')
s=s.replace('fmt.Printf("서버 기동: %s  group=%s\\n", addr, group)',
            'fmt.Printf("서버 기동: %s  group=%s  인증서=%dB (목표 %dB)\\n", addr, group, actualSize, certSize)')

# 4) main 에 -certsize 플래그
s=s.replace('\tn := flag.Int("n", 5, "반복 횟수")',
            '\tn := flag.Int("n", 5, "반복 횟수")\n\tcertSize := flag.Int("certsize", 0, "서버 인증서 목표 크기(B)")')
s=s.replace('\t\trunServer(*addr, *group)','\t\trunServer(*addr, *group, *certSize)')

open(p,'w').write(s); print("패치 완료")
```

A.6 patch2_initpkt.py — Initial 패킷 크기 패치 (29 줄)

클라이언트 Initial 패킷 크기를 조작해 패딩 인과를 분리 검증한다. 6.8 절 참조.

```
# qamp.go 패치 2 — 클라이언트 Initial 패킷 크기 조절 (-initpkt 플래그)
import re
p='qamp.go'; s=open(p).read()

# 클라이언트 함수 시그니처에 ips 추가
s=s.replace('func runClient(addr, group string, n int) {',
            'func runClient(addr, group string, n, ips int) {')
# Dial 의 quic.Config 에 InitialPacketSize 주입
s=s.replace('''        conn, err := tr.Dial(ctx, raddr, tlsConf, &quic.Config{
            HandshakeIdleTimeout:    5 * time.Second,''',
'''        qc := &quic.Config{
            HandshakeIdleTimeout:    5 * time.Second,''')
s=s.replace('''            MaxIdleTimeout:          30 * time.Second,
            DisablePathMTUDiscovery: true,
        })
        el := float64(time.Since(t0).Microseconds()) / 1000.0''',
'''            MaxIdleTimeout:          30 * time.Second,
            DisablePathMTUDiscovery: true,
        }
        if ips > 0 {
            qc.InitialPacketSize = uint16(ips)
        }
        conn, err := tr.Dial(ctx, raddr, tlsConf, qc)
        el := float64(time.Since(t0).Microseconds()) / 1000.0''')
# main 에 플래그
s=s.replace('\tcertSize := flag.Int("certsize", 0, "서버 인증서 목표 크기(B)")',
            '\tcertSize := flag.Int("certsize", 0, "서버 인증서 목표 크기(B)")\n\tips := flag.Int("initpkt", 0, "클라이언트
Initial 패킷 크기(B, 1200~1472). 0이면 기본")')
s=s.replace('\t\trunClient(*addr, *group, *n)','\t\trunClient(*addr, *group, *n, *ips)')
open(p,'w').write(s); print("패치 완료")
```

A.7 amp_sweep.sh — 증폭 임계 스윕 (26 줄)

인증서 크기를 100 B 간격으로 훑어 전환 임계를 특정한다. 6.3 절 참조.

```
#!/usr/bin/env bash
# 증폭 임계 정밀 스윕 — 중첩 netns + veth + 양방향 대칭 netem(각 25ms = RTT 50ms)
cd ~/Desktop/workspace/34/"1311 신레오"/quic
cp qamp /tmp/qamp
unshare -Urnm bash -c '
mount -t tmpfs none /run 2>/dev/null; mkdir -p /run/netns
ip netns add peer
```

```
ip link add v0 type veth peer name v1
ip link set v1 netns peer
ip addr add 10.9.0.1/24 dev v0; ip link set v0 mtu 1500; ip link set v0 up
ip netns exec peer sh -c "ip addr add 10.9.0.2/24 dev v1; ip link set v1 mtu 1500; ip link set v1 up"
tc qdisc add dev v0 root netem delay 25ms
ip netns exec peer tc qdisc add dev v1 root netem delay 25ms
echo "certsize,group,p50_ms,sent_pkts,sent_bytes,recv_pkts,recv_bytes"
for cs in 2900 3000 3100 3200 3300 3400 3500 4800 4900 5000 5100 5200 5300 5400 5500 5600 5700 5800 5900 6000; do
for g in X25519 X25519MLKEM768; do
  ip netns exec peer /tmp/qamp -mode server -addr 10.9.0.2:44494 -group $g -certsize $cs >/dev/null 2>&1 &
  SRV=$!; sleep 1
  OUT=$(timeout 120 /tmp/qamp -mode client -addr 10.9.0.2:44494 -group $g -n 30 2>/dev/null)
  P50=$(echo "$OUT" | grep "HCT=" | awk -F"HCT=" "{split(\$2,a,\"ms\"); print a[1]}" | sort -n | awk "{v[NR]=\$1} END
{printf \"%.1f\", v[int(NR/2)]}")
  AVG=$(echo "$OUT" | grep "평균:" | sed "s/[^0-9. ]//g")
  echo "$cs,$g,$P50,$AVG"
  kill $SRV 2>/dev/null; sleep 1
done
done
'
```

데이터 전문

실험에서 산출한 원시 데이터다. TLS 스윕 요약 CSV 두 개(1 차 120 행, 2 차 250 행)와 증폭 실험의 원시 출력을 싣는다. 개별 조건별 반복 원본은 엑셀 산출물에 있다.

B.1 TLS 1 차 스윕 요약 (120 조건)

경고: 1 차 스윕은 loopback MTU 65,536 미적용으로 손실 노출이 생성되지 않아 무효다. 대조·재해석 용도로만 싣는다(4.5 절). 열: model, rtt_ms, loss_pct, group, n, fail, fail_rate_pct, mean_ms, sd_ms, p50_ms, p95_ms, p99_ms.

model	rtt_ms	loss_pct	group	n	fail	fail_rate_pct	mean_ms	sd_ms	p50_ms	p95_ms	p99_ms
uniform	0	0	X25519	200	0	0.00	1.575	0.088	1.566	1.704	1.936
uniform	0	0	MLKEM512	200	0	0.00	1.434	0.111	1.410	1.689	1.972
uniform	0	0	MLKEM768	200	0	0.00	1.610	0.151	1.585	1.961	2.115
uniform	0	0	MLKEM1024	200	0	0.00	1.789	0.098	1.774	1.907	2.257
uniform	0	0	X25519MLKEM768	200	0	0.00	1.909	0.216	1.981	2.172	2.285
uniform	0	1	X25519	800	0	0.00	30.198	167.704	1.643	3.467	1019.991
uniform	0	1	MLKEM512	800	0	0.00	21.243	130.160	1.571	2.971	1024.963
uniform	0	1	MLKEM768	800	0	0.00	26.878	148.338	1.645	2.768	1025.303
uniform	0	1	MLKEM1024	800	0	0.00	34.777	172.269	1.896	4.246	1044.646
uniform	0	1	X25519MLKEM768	800	0	0.00	21.384	124.024	2.187	3.534	1010.869
uniform	0	2	X25519	800	0	0.00	76.312	267.538	1.837	1009.844	1061.333
uniform	0	2	MLKEM512	800	0	0.00	49.721	204.442	1.581	208.654	1056.978
uniform	0	2	MLKEM768	800	0	0.00	53.032	210.435	2.046	209.991	1053.926
uniform	0	2	MLKEM1024	800	0	0.00	59.454	232.300	2.018	210.062	1055.632
uniform	0	2	X25519MLKEM768	800	0	0.00	52.478	201.885	2.733	210.583	1045.763
uniform	0	5	X25519	800	0	0.00	153.466	461.094	2.041	1037.597	2072.153
uniform	0	5	MLKEM512	800	0	0.00	138.297	371.049	2.182	1036.668	1231.827
uniform	0	5	MLKEM768	800	0	0.00	172.806	516.661	2.272	1045.318	2087.876
uniform	0	5	MLKEM1024	800	0	0.00	138.451	387.273	2.014	1023.923	1249.305
uniform	0	5	X25519MLKEM768	800	0	0.00	132.725	356.801	2.290	1024.434	1223.235
uniform	10	0	X25519	200	0	0.00	22.225	0.050	22.223	22.311	22.360
uniform	10	0	MLKEM512	200	0	0.00	22.180	0.179	22.158	22.283	23.061
uniform	10	0	MLKEM768	200	0	0.00	22.338	0.191	22.308	22.667	22.992
uniform	10	0	MLKEM1024	200	0	0.00	22.589	0.436	22.426	23.578	23.823
uniform	10	0	X25519MLKEM768	200	0	0.00	22.973	0.375	22.833	23.877	24.188
uniform	10	1	X25519	800	0	0.00	40.906	122.923	22.239	22.977	1030.155
uniform	10	1	MLKEM512	800	0	0.00	55.696	197.008	22.240	39.241	1059.686
uniform	10	1	MLKEM768	800	0	0.00	43.875	147.351	22.357	23.933	1060.801
uniform	10	1	MLKEM1024	800	0	0.00	46.517	142.345	22.548	27.788	1040.139

uniform	10	1	X25519MLKEM768	800	0	0.00	44.897	132.919	22.767	27.690	1039.608
uniform	10	2	X25519	800	0	0.00	77.472	227.319	22.350	249.297	1078.542
uniform	10	2	MLKEM512	800	0	0.00	83.168	239.450	22.190	250.692	1076.952
uniform	10	2	MLKEM768	800	0	0.00	63.708	189.501	22.321	246.983	1063.083
uniform	10	2	MLKEM1024	800	0	0.00	81.371	230.907	22.540	261.797	1069.531
uniform	10	2	X25519MLKEM768	800	0	0.00	62.825	186.163	22.866	246.638	1063.629
uniform	10	5	X25519	800	0	0.00	172.732	377.951	22.331	1063.465	1245.872
uniform	10	5	MLKEM512	800	0	0.00	156.265	372.296	22.196	1056.529	1285.355
uniform	10	5	MLKEM768	800	0	0.00	149.786	363.020	22.365	1048.947	1273.623
uniform	10	5	MLKEM1024	800	0	0.00	163.654	383.378	22.607	1057.713	1305.604
uniform	10	5	X25519MLKEM768	800	0	0.00	165.421	377.093	23.145	1050.723	1292.170
uniform	50	0	X25519	200	0	0.00	102.542	0.423	102.352	103.424	103.767
uniform	50	0	MLKEM512	200	0	0.00	102.661	0.679	102.301	103.830	104.548
uniform	50	0	MLKEM768	200	0	0.00	102.401	0.281	102.320	103.080	103.336
uniform	50	0	MLKEM1024	200	0	0.00	102.622	0.359	102.488	103.359	103.888
uniform	50	0	X25519MLKEM768	200	0	0.00	103.371	0.713	103.026	104.658	105.267
uniform	50	1	X25519	800	0	0.00	135.315	165.771	102.344	117.894	1127.281
uniform	50	1	MLKEM512	800	0	0.00	127.243	177.541	102.147	102.653	1114.414
uniform	50	1	MLKEM768	800	0	0.00	131.680	176.112	102.345	104.331	1127.522
uniform	50	1	MLKEM1024	800	0	0.00	123.538	136.669	102.560	103.797	1120.141
uniform	50	1	X25519MLKEM768	800	0	0.00	134.215	166.765	102.860	104.549	1124.649
uniform	50	2	X25519	800	0	0.00	151.894	228.463	102.299	364.680	1135.108
uniform	50	2	MLKEM512	800	0	0.00	148.385	195.978	102.180	406.849	1137.628
uniform	50	2	MLKEM768	800	0	0.00	167.750	250.757	102.364	426.483	1147.023
uniform	50	2	MLKEM1024	800	0	0.00	143.067	181.692	102.600	406.262	1143.051
uniform	50	2	X25519MLKEM768	800	0	0.00	154.523	212.281	102.897	409.060	1133.375
uniform	50	5	X25519	800	0	0.00	241.139	381.144	102.335	1125.190	1419.581
uniform	50	5	MLKEM512	800	0	0.00	250.733	376.416	102.482	1131.347	1404.122
uniform	50	5	MLKEM768	800	0	0.00	239.587	374.008	102.499	1125.928	1466.235
uniform	50	5	MLKEM1024	800	0	0.00	252.508	396.725	102.583	1135.566	1514.360
uniform	50	5	X25519MLKEM768	800	0	0.00	278.640	490.940	102.922	1134.631	2176.306
burst	0	0	X25519	200	0	0.00	1.601	0.113	1.581	1.935	2.033
burst	0	0	MLKEM512	200	0	0.00	1.460	0.069	1.466	1.541	1.682
burst	0	0	MLKEM768	200	0	0.00	1.545	0.190	1.611	1.682	2.214
burst	0	0	MLKEM1024	200	0	0.00	1.716	0.202	1.816	1.891	2.316
burst	0	0	X25519MLKEM768	200	0	0.00	2.133	0.078	2.145	2.199	2.227
burst	0	1	X25519	800	0	0.00	46.914	221.285	1.649	204.178	1046.290
burst	0	1	MLKEM512	800	0	0.00	48.361	233.560	1.490	204.175	1049.189

burst	0	1	MLKEM768	800	0	0.00	74.820	376.391	1.657	415.942	1056.777
burst	0	1	MLKEM1024	800	0	0.00	62.131	522.027	1.937	207.610	1054.536
burst	0	1	X25519MLKEM768	800	0	0.00	58.363	265.420	2.691	221.856	1060.130
burst	0	2	X25519	800	0	0.00	163.528	1321.146	1.750	871.331	2369.700
burst	0	2	MLKEM512	800	0	0.00	271.112	2111.306	1.553	1017.152	4098.246
burst	0	2	MLKEM768	800	0	0.00	168.954	1358.757	1.758	1021.124	3053.356
burst	0	2	MLKEM1024	800	0	0.00	181.033	1238.471	1.957	1029.903	2054.374
burst	0	2	X25519MLKEM768	800	0	0.00	119.850	464.921	2.636	1021.578	1693.738
burst	0	5	X25519	800	0	0.00	387.980	1501.805	2.212	1257.770	4867.682
burst	0	5	MLKEM512	800	0	0.00	425.735	1694.470	2.034	1439.533	6525.773
burst	0	5	MLKEM768	0	0	0					
burst	0	5	MLKEM1024	0	0	0					
burst	0	5	X25519MLKEM768	800	0	0.00	1006.582	10576.744	2.704	2086.946	11223.001
burst	10	0	X25519	200	0	0.00	22.474	0.450	22.309	23.480	23.980
burst	10	0	MLKEM512	200	0	0.00	22.403	0.348	22.232	23.007	23.275
burst	10	0	MLKEM768	200	0	0.00	22.568	0.442	22.355	23.514	23.757
burst	10	0	MLKEM1024	200	0	0.00	22.556	0.263	22.496	23.166	23.800
burst	10	0	X25519MLKEM768	200	0	0.00	23.078	0.419	22.901	23.952	24.438
burst	10	1	X25519	800	0	0.00	75.743	270.486	22.251	246.228	1191.596
burst	10	1	MLKEM512	800	0	0.00	78.186	249.314	22.165	251.471	1072.706
burst	10	1	MLKEM768	800	0	0.00	148.095	1471.330	22.322	466.032	1071.376
burst	10	1	MLKEM1024	800	0	0.00	302.458	3444.183	22.505	260.530	3220.623
burst	10	1	X25519MLKEM768	800	0	0.00	96.621	644.597	22.832	247.183	1080.420
burst	10	2	X25519	800	0	0.00	151.930	689.617	22.195	1033.572	1788.846
burst	10	2	MLKEM512	800	0	0.00	120.342	564.859	22.189	1024.070	1088.757
burst	10	2	MLKEM768	800	0	0.00	114.772	418.830	22.324	876.873	1262.183
burst	10	2	MLKEM1024	800	0	0.00	130.067	409.754	22.574	1039.710	1553.005
burst	10	2	X25519MLKEM768	800	0	0.00	158.050	1156.507	22.876	882.616	1313.883
burst	10	5	X25519	800	0	0.00	888.801	11742.059	22.607	1761.757	8057.713
burst	10	5	MLKEM512	799	1	0.12	955.803	12299.749	22.807	2098.472	7700.310
burst	10	5	MLKEM768	800	0	0.00	708.365	4491.245	22.919	2143.486	10087.568
burst	10	5	MLKEM1024	800	0	0.00	500.427	2009.187	23.219	1577.932	8018.233
burst	10	5	X25519MLKEM768	800	0	0.00	1814.940	21072.154	23.728	2127.456	17489.760
burst	50	0	X25519	200	0	0.00	102.560	0.383	102.389	103.255	103.584
burst	50	0	MLKEM512	200	0	0.00	102.405	0.421	102.234	103.108	103.692
burst	50	0	MLKEM768	200	0	0.00	102.257	0.121	102.249	102.336	102.524
burst	50	0	MLKEM1024	20	0	0.00	102.6	0.33	102.	103.	103.

				0			38	5	494	456	681
burst	50	0	X25519MLKEM768	200	0	0.00	103.168	0.604	102.896	104.487	105.074
burst	50	1	X25519	800	0	0.00	412.045	3916.915	102.327	406.582	2645.442
burst	50	1	MLKEM512	800	0	0.00	161.980	333.249	102.237	405.413	1145.572
burst	50	1	MLKEM768	800	0	0.00	812.436	18308.652	102.529	386.827	1154.193
burst	50	1	MLKEM1024	800	0	0.00	162.079	381.894	102.598	371.587	1155.993
burst	50	1	X25519MLKEM768	800	0	0.00	156.213	297.007	102.873	358.918	1159.239
burst	50	2	X25519	800	0	0.00	217.965	470.459	102.312	1116.552	2135.716
burst	50	2	MLKEM512	800	0	0.00	319.258	1230.827	102.241	1130.565	4071.503
burst	50	2	MLKEM768	800	0	0.00	462.184	4633.740	102.400	1126.808	3162.878
burst	50	2	MLKEM1024	800	0	0.00	422.908	3202.454	103.081	1138.883	4512.773
burst	50	2	X25519MLKEM768	800	0	0.00	354.248	1743.614	103.199	1122.939	5846.685
burst	50	5	X25519	800	0	0.00	688.496	3239.910	102.541	2371.437	7953.372
burst	50	5	MLKEM512	800	0	0.00	1232.805	9856.816	102.405	3162.298	10035.835
burst	50	5	MLKEM768	800	0	0.00	526.852	1821.601	102.438	1985.669	5629.939
burst	50	5	MLKEM1024	800	0	0.00	708.495	4155.693	102.724	1471.662	5263.130
burst	50	5	X25519MLKEM768	800	0	0.00	888.733	6136.919	103.100	2543.943	9384.543

B.2 TLS 2 차 스윕 요약 (250 조건)

veth + MTU 1500 + 타임아웃으로 재수행한 유효 데이터. 열은 1 차에 tmo(타임아웃 건수)가 추가된 13 열이다. 실패율 분석의 근거다(4.6 절).

model	rtt_ms	loss_pct	group	n	fail	tmo	fail_rate_pct	mean_ms	sd_ms	p50_ms	p95_ms	p99_ms
uniform	0	0	X25519	250	0	0	0.00	1.290	0.174	1.250	1.604	1.818
uniform	0	0	MLKEM512	250	0	0	0.00	1.400	0.145	1.390	1.719	1.831
uniform	0	0	MLKEM768	250	0	0	0.00	1.110	0.132	1.047	1.406	1.612
uniform	0	0	MLKEM1024	250	0	0	0.00	1.995	0.210	1.951	2.328	2.641
uniform	0	0	X25519MLKEM768	250	0	0	0.00	2.467	0.324	2.456	3.007	3.307
uniform	0	0.5	X25519	2000	0	0	0.00	17.515	115.120	2.627	3.602	1021.220
uniform	0	0.5	MLKEM512	2000	0	0	0.00	15.850	114.760	2.482	3.243	1013.915
uniform	0	0.5	MLKEM768	2000	0	0	0.00	17.995	129.682	2.716	3.591	1017.760
uniform	0	0.5	MLKEM1024	2000	0	0	0.00	11.400	92.514	3.000	3.679	7.951
uniform	0	0.5	X25519MLKEM768	2000	0	0	0.00	15.456	109.670	3.123	4.095	1005.402
uniform	0	1	X25519	2000	0	0	0.00	27.720	167.946	2.631	3.539	1031.344
uniform	0	1	MLKEM512	2000	0	0	0.00	25.099	150.572	2.516	3.404	1033.345
uniform	0	1	MLKEM768	2000	0	0	0.00	23.174	155.620	2.749	3.679	1028.964
uniform	0	1	MLKEM1024	2000	0	0	0.00	27.822	157.887	3.053	4.051	1044.450
uniform	0	1	X25519MLKEM768	2000	0	0	0.00	31.216	173.885	3.453	4.957	1046.826

uniform	0	2	X25519	2000	0	0	0.00	49.638	209.042	2.662	208.859	1053.604
uniform	0	2	MLKEM512	2000	0	0	0.00	53.842	233.500	2.548	210.708	1056.500
uniform	0	2	MLKEM768	2000	0	0	0.00	51.250	211.634	2.761	210.121	1058.499
uniform	0	2	MLKEM1024	1999	1	1	0.05	55.499	233.959	3.078	213.504	1059.285
uniform	0	2	X25519MLKEM768	2000	0	0	0.00	50.657	212.632	3.478	210.713	1059.612
uniform	0	5	X25519	2000	0	0	0.00	131.931	351.013	2.759	1029.207	1220.736
uniform	0	5	MLKEM512	2000	0	0	0.00	125.423	364.545	2.651	1033.475	1260.454
uniform	0	5	MLKEM768	2000	0	0	0.00	126.093	356.454	2.863	1028.669	1261.350
uniform	0	5	MLKEM1024	1996	4	4	0.20	128.742	396.700	3.154	1035.629	2038.659
uniform	0	5	X25519MLKEM768	1999	1	1	0.05	132.732	378.741	3.591	1036.127	1272.126
uniform	10	0	X25519	250	0	0	0.00	22.887	0.396	22.879	23.472	23.805
uniform	10	0	MLKEM512	250	0	0	0.00	22.773	0.439	22.784	23.407	23.774
uniform	10	0	MLKEM768	250	0	0	0.00	22.906	0.390	22.924	23.444	23.676
uniform	10	0	MLKEM1024	250	0	0	0.00	23.286	0.591	23.332	24.063	24.897
uniform	10	0	X25519MLKEM768	250	0	0	0.00	23.655	0.545	23.720	24.317	24.538
uniform	10	0.5	X25519	2000	0	0	0.00	37.682	114.460	22.906	23.684	1035.682
uniform	10	0.5	MLKEM512	2000	0	0	0.00	35.337	109.331	22.772	23.519	262.686
uniform	10	0.5	MLKEM768	2000	0	0	0.00	35.584	109.301	22.984	23.911	1037.747
uniform	10	0.5	MLKEM1024	2000	0	0	0.00	36.476	116.331	23.314	24.134	1031.305
uniform	10	0.5	X25519MLKEM768	2000	0	0	0.00	32.235	79.981	23.788	24.817	240.585
uniform	10	1	X25519	2000	0	0	0.00	50.637	158.278	22.919	24.390	1057.301
uniform	10	1	MLKEM512	2000	0	0	0.00	51.458	175.706	22.792	27.989	1064.122
uniform	10	1	MLKEM768	2000	0	0	0.00	50.875	174.934	23.040	24.486	1064.980
uniform	10	1	MLKEM1024	1999	1	1	0.05	57.984	191.088	23.353	29.116	1069.512
uniform	10	1	X25519MLKEM768	2000	0	0	0.00	46.274	156.053	23.748	25.404	1052.938
uniform	10	2	X25519	2000	0	0	0.00	72.128	223.993	22.950	245.805	1067.960
uniform	10	2	MLKEM512	2000	0	0	0.00	74.233	213.848	22.824	259.347	1075.735
uniform	10	2	MLKEM768	2000	0	0	0.00	78.462	221.860	23.066	260.604	1077.178
uniform	10	2	MLKEM1024	1999	1	1	0.05	76.244	249.389	23.395	262.217	1083.107
uniform	10	2	X25519MLKEM768	2000	0	0	0.00	75.636	229.791	23.785	254.753	1080.923
uniform	10	5	X25519	2000	0	0	0.00	152.232	378.599	23.024	1054.287	1525.737
uniform	10	5	MLKEM512	1998	2	2	0.10	166.760	397.396	22.922	1068.690	1853.704
uniform	10	5	MLKEM768	2000	0	0	0.00	158.432	380.465	23.147	1062.272	1300.622
uniform	10	5	MLKEM1024	1989	11	11	0.54	179.243	433.852	23.473	1079.592	2099.779
uniform	10	5	X25519MLKEM768	1999	1	1	0.05	172.916	403.691	23.933	1068.249	1496.412
uniform	50	0	X25519	250	0	0	0.00	103.124	0.363	103.170	103.634	103.942
uniform	50	0	MLKEM512	250	0	0	0.00	103.122	0.373	103.102	103.697	104.106
uniform	50	0	MLKEM768	250	0	0	0.00	103.301	0.382	103.296	103.789	104.561

uniform	50	0	MLKEM1024	250	0	0	0.00	103.584	0.418	103.608	104.125	104.474
uniform	50	0	X25519MLKEM768	250	0	0	0.00	104.046	0.493	104.069	104.742	105.015
uniform	50	0.5	X25519	2000	0	0	0.00	114.504	97.547	103.187	103.766	360.325
uniform	50	0.5	MLKEM512	2000	0	0	0.00	119.395	122.205	103.076	103.688	1122.978
uniform	50	0.5	MLKEM768	2000	0	0	0.00	120.395	123.487	103.340	104.065	1119.760
uniform	50	0.5	MLKEM1024	2000	0	0	0.00	118.451	111.211	103.627	104.471	628.545
uniform	50	0.5	X25519MLKEM768	2000	0	0	0.00	116.458	103.800	104.089	104.886	468.973
uniform	50	1	X25519	2000	0	0	0.00	128.981	159.214	103.205	104.812	1133.308
uniform	50	1	MLKEM512	2000	0	0	0.00	129.913	165.309	103.106	104.058	1142.104
uniform	50	1	MLKEM768	2000	0	0	0.00	130.540	155.724	103.329	104.826	1150.864
uniform	50	1	MLKEM1024	2000	0	0	0.00	128.761	142.658	103.648	118.388	1128.614
uniform	50	1	X25519MLKEM768	2000	0	0	0.00	128.462	152.317	104.107	105.037	1142.548
uniform	50	2	X25519	2000	0	0	0.00	147.622	199.038	103.228	360.280	1133.389
uniform	50	2	MLKEM512	2000	0	0	0.00	165.848	254.971	103.113	468.941	1175.645
uniform	50	2	MLKEM768	2000	0	0	0.00	156.195	231.644	103.344	463.011	1170.507
uniform	50	2	MLKEM1024	2000	0	0	0.00	155.769	219.207	103.682	466.337	1169.197
uniform	50	2	X25519MLKEM768	2000	0	0	0.00	159.223	231.314	104.143	465.325	1170.343
uniform	50	5	X25519	2000	0	0	0.00	254.317	411.759	103.296	1134.105	2134.701
uniform	50	5	MLKEM512	2000	0	0	0.00	260.629	409.857	103.221	1161.265	1691.067
uniform	50	5	MLKEM768	1998	2	2	0.10	261.898	418.228	103.435	1155.334	2013.405
uniform	50	5	MLKEM1024	1993	7	7	0.34	264.235	451.658	103.749	1166.030	2210.973
uniform	50	5	X25519MLKEM768	1999	1	1	0.05	251.897	407.824	104.206	1154.847	2136.303
uniform	100	0	X25519	125	0	0	0.00	203.205	0.334	203.243	203.621	203.754
uniform	100	0	MLKEM512	125	0	0	0.00	203.201	0.430	203.145	204.150	204.355
uniform	100	0	MLKEM768	125	0	0	0.00	203.319	0.375	203.345	203.914	204.065
uniform	100	0	MLKEM1024	125	0	0	0.00	203.653	0.325	203.660	204.056	204.291
uniform	100	0	X25519MLKEM768	125	0	0	0.00	204.064	0.443	204.110	204.694	205.188
uniform	100	0.5	X25519	1000	0	0	0.00	217.330	108.362	203.263	203.888	617.655
uniform	100	0.5	MLKEM512	1000	0	0	0.00	215.262	98.243	203.158	203.736	713.993
uniform	100	0.5	MLKEM768	1000	0	0	0.00	219.472	151.988	203.399	204.054	717.956
uniform	100	0.5	MLKEM1024	1000	0	0	0.00	216.349	102.291	203.741	204.350	748.670
uniform	100	0.5	X25519MLKEM768	1000	0	0	0.00	217.643	114.083	204.146	204.794	1208.429
uniform	100	1	X25519	1000	0	0	0.00	223.620	123.467	203.291	204.174	1205.139
uniform	100	1	MLKEM512	1000	0	0	0.00	227.558	145.984	203.190	204.066	1227.603
uniform	100	1	MLKEM768	1000	0	0	0.00	240.519	190.723	203.382	230.100	1244.738
uniform	100	1	MLKEM1024	1000	0	0	0.00	241.233	179.459	203.738	509.534	1244.289
uniform	100	1	X25519MLKEM768	1000	0	0	0.00	239.155	196.301	204.171	207.367	1230.341
uniform	100	2	X25519	1000	0	0	0.00	259.106	236.321	203.293	508.451	1250.948

uniform	100	2	MLKEM512	1000	0	0	0.00	265.128	233.164	203.187	717.439	1250.740
uniform	100	2	MLKEM768	1000	0	0	0.00	266.972	245.139	203.427	714.720	1304.444
uniform	100	2	MLKEM1024	1000	0	0	0.00	261.301	215.441	203.736	715.400	1265.985
uniform	100	2	X25519MLKEM768	1000	0	0	0.00	259.612	232.210	204.187	712.187	1247.071
uniform	100	5	X25519	1000	0	0	0.00	358.396	422.916	203.411	1238.447	2252.065
uniform	100	5	MLKEM512	1000	0	0	0.00	357.280	381.211	203.277	1235.029	2228.279
uniform	100	5	MLKEM768	1000	0	0	0.00	363.973	403.358	203.472	1240.216	1717.404
uniform	100	5	MLKEM1024	1000	0	0	0.00	373.574	433.111	203.821	1251.251	2268.050
uniform	100	5	X25519MLKEM768	1000	0	0	0.00	371.896	404.267	204.389	1244.116	1882.703
uniform	200	0	X25519	125	0	0	0.00	403.164	0.338	403.226	403.681	403.954
uniform	200	0	MLKEM512	125	0	0	0.00	403.047	0.288	403.126	403.371	403.580
uniform	200	0	MLKEM768	125	0	0	0.00	403.553	0.542	403.500	404.605	405.029
uniform	200	0	MLKEM1024	125	0	0	0.00	403.733	0.594	403.721	404.328	405.692
uniform	200	0	X25519MLKEM768	125	0	0	0.00	404.111	0.350	404.123	404.620	404.974
uniform	200	0.5	X25519	1000	0	0	0.00	418.431	110.834	403.261	404.157	1049.635
uniform	200	0.5	MLKEM512	1000	0	0	0.00	426.626	151.549	403.177	403.890	1457.258
uniform	200	0.5	MLKEM768	1000	0	0	0.00	417.825	112.424	403.392	404.183	1021.824
uniform	200	0.5	MLKEM1024	1000	0	0	0.00	419.755	128.758	403.707	404.486	1427.804
uniform	200	0.5	X25519MLKEM768	1000	0	0	0.00	422.053	126.874	404.156	405.048	1417.659
uniform	200	1	X25519	1000	0	0	0.00	436.660	166.938	403.291	404.445	1427.176
uniform	200	1	MLKEM512	1000	0	0	0.00	432.270	167.691	403.190	404.565	1446.262
uniform	200	1	MLKEM768	1000	0	0	0.00	438.635	181.778	403.414	404.692	1448.760
uniform	200	1	MLKEM1024	1000	0	0	0.00	448.154	237.867	403.739	405.038	1613.240
uniform	200	1	X25519MLKEM768	1000	0	0	0.00	442.029	185.831	404.180	456.116	1456.293
uniform	200	2	X25519	1000	0	0	0.00	465.731	229.021	403.302	1045.886	1436.853
uniform	200	2	MLKEM512	1000	0	0	0.00	475.356	263.897	403.207	1047.766	1477.496
uniform	200	2	MLKEM768	1000	0	0	0.00	472.192	251.945	403.409	1050.017	1466.858
uniform	200	2	MLKEM1024	999	1	1	0.10	493.088	360.141	403.773	1411.595	2162.245
uniform	200	2	X25519MLKEM768	1000	0	0	0.00	472.498	258.699	404.185	1065.612	1617.769
uniform	200	5	X25519	999	1	1	0.10	612.323	487.576	403.412	1434.399	2485.734
uniform	200	5	MLKEM512	998	2	2	0.19	640.611	523.526	403.264	1604.068	2475.113
uniform	200	5	MLKEM768	999	1	1	0.10	610.478	511.677	403.491	1471.705	2543.902
uniform	200	5	MLKEM1024	993	7	7	0.67	659.534	608.540	403.838	1607.501	3233.250
uniform	200	5	X25519MLKEM768	999	1	1	0.10	627.694	500.362	404.259	1471.971	2449.008
burst	0	0	X25519	250	0	0	0.00	1.528	0.392	1.617	2.127	2.437
burst	0	0	MLKEM512	250	0	0	0.00	1.214	0.188	1.199	1.554	1.790
burst	0	0	MLKEM768	250	0	0	0.00	1.542	0.149	1.512	1.846	1.961
burst	0	0	MLKEM1024	250	0	0	0.00	1.655	0.288	1.669	2.284	2.544

burst	0	0	X25519MLKEM768	250	0	0	0.00	2.184	0.472	2.315	2.864	3.276
burst	0	0.5	X25519	1998	3	3	0.15	27.538	168.890	2.627	3.798	1035.322
burst	0	0.5	MLKEM512	2000	0	0	0.00	25.420	170.919	2.485	3.450	1023.245
burst	0	0.5	MLKEM768	1997	3	3	0.15	25.610	182.057	2.708	3.624	1032.360
burst	0	0.5	MLKEM1024	1999	1	1	0.05	22.303	147.628	3.029	3.792	1022.219
burst	0	0.5	X25519MLKEM768	1999	1	1	0.05	20.581	142.523	3.421	4.313	1019.464
burst	0	1	X25519	2000	0	0	0.00	49.856	235.204	2.651	208.170	1054.481
burst	0	1	MLKEM512	1999	1	1	0.05	57.315	270.983	2.553	211.208	1063.444
burst	0	1	MLKEM768	1996	4	4	0.20	48.410	245.560	2.742	209.587	1053.461
burst	0	1	MLKEM1024	1991	9	9	0.44	43.785	207.404	3.057	6.927	1051.600
burst	0	1	X25519MLKEM768	1999	1	1	0.05	54.540	268.908	3.478	210.557	1060.004
burst	0	2	X25519	1993	7	7	0.34	104.869	331.698	2.673	1020.259	1066.567
burst	0	2	MLKEM512	1994	6	6	0.29	96.086	336.345	2.542	1020.264	1316.505
burst	0	2	MLKEM768	1993	7	7	0.34	86.818	321.942	2.813	1015.632	1258.122
burst	0	2	MLKEM1024	1988	13	13	0.63	99.724	375.055	3.119	1022.635	1555.094
burst	0	2	X25519MLKEM768	1995	5	5	0.24	105.963	391.220	3.523	1024.002	1976.766
burst	0	5	X25519	1966	35	35	1.71	275.081	663.457	2.896	1232.084	3351.593
burst	0	5	MLKEM512	1963	39	39	1.90	271.877	665.604	2.794	1271.295	3174.745
burst	0	5	MLKEM768	1969	32	32	1.56	272.128	657.769	2.995	1226.939	3170.536
burst	0	5	MLKEM1024	1957	49	49	2.39	264.960	642.440	3.349	1260.947	3160.072
burst	0	5	X25519MLKEM768	1966	34	34	1.66	272.283	660.556	3.683	1245.787	3265.416
burst	10	0	X25519	250	0	0	0.00	22.955	0.369	22.924	23.517	23.780
burst	10	0	MLKEM512	250	0	0	0.00	22.744	0.365	22.764	23.180	23.443
burst	10	0	MLKEM768	250	0	0	0.00	23.068	0.350	23.051	23.559	23.869
burst	10	0	MLKEM1024	250	0	0	0.00	23.352	0.410	23.349	23.949	24.236
burst	10	0	X25519MLKEM768	250	0	0	0.00	23.743	0.415	23.740	24.250	24.603
burst	10	0.5	X25519	2000	0	0	0.00	48.676	183.454	22.916	24.072	1036.606
burst	10	0.5	MLKEM512	2000	0	0	0.00	47.090	161.556	22.767	23.747	1045.241
burst	10	0.5	MLKEM768	1997	3	3	0.15	56.265	233.331	23.016	24.138	1075.348
burst	10	0.5	MLKEM1024	1998	2	2	0.10	47.505	169.349	23.334	24.749	1046.915
burst	10	0.5	X25519MLKEM768	1999	1	1	0.05	47.203	150.618	23.766	25.248	1048.348
burst	10	1	X25519	1999	1	1	0.05	76.294	259.240	22.921	239.801	1075.874
burst	10	1	MLKEM512	1999	3	3	0.15	72.575	239.760	22.783	247.595	1077.697
burst	10	1	MLKEM768	1998	2	2	0.10	74.578	257.896	23.033	256.349	1085.867
burst	10	1	MLKEM1024	1998	2	2	0.10	74.937	290.036	23.358	261.563	1075.617
burst	10	1	X25519MLKEM768	1998	2	2	0.10	68.689	265.963	23.682	238.655	1073.233
burst	10	2	X25519	1991	9	9	0.44	132.848	385.700	22.975	1041.620	1895.346
burst	10	2	MLKEM512	1993	7	7	0.34	133.908	400.533	22.881	1047.679	2066.277

burst	10	2	MLKEM768	1994	6	6	0.29	114.201	336.012	23.080	1031.916	1529.043
burst	10	2	MLKEM1024	1989	11	11	0.54	137.248	404.202	23.398	1056.082	1852.565
burst	10	2	X25519MLKEM768	1988	12	12	0.59	141.110	410.467	23.822	1052.406	2073.948
burst	10	5	X25519	1964	36	36	1.76	331.997	716.417	23.191	1685.141	3480.474
burst	10	5	MLKEM512	1976	24	24	1.17	273.366	651.972	23.019	1087.948	3307.679
burst	10	5	MLKEM768	1958	43	43	2.10	305.697	650.117	23.274	1552.654	3321.089
burst	10	5	MLKEM1024	1952	58	58	2.83	302.060	695.717	23.575	1320.579	3388.138
burst	10	5	X25519MLKEM768	1972	29	29	1.41	311.182	692.339	24.045	1300.245	3474.876
burst	50	0	X25519	250	0	0	0.00	103.131	0.356	103.153	103.577	104.055
burst	50	0	MLKEM512	250	0	0	0.00	103.144	0.358	103.103	103.871	104.153
burst	50	0	MLKEM768	250	0	0	0.00	103.209	0.389	103.274	103.733	103.958
burst	50	0	MLKEM1024	250	0	0	0.00	103.608	0.401	103.633	104.169	104.434
burst	50	0	X25519MLKEM768	250	0	0	0.00	104.086	0.391	104.095	104.633	104.952
burst	50	0.5	X25519	1999	1	1	0.05	127.571	187.937	103.226	104.077	1117.454
burst	50	0.5	MLKEM512	2000	0	0	0.00	133.247	186.129	103.131	104.000	1151.343
burst	50	0.5	MLKEM768	1996	4	4	0.20	130.019	231.671	103.386	104.306	1129.829
burst	50	0.5	MLKEM1024	1998	2	2	0.10	136.637	213.548	103.682	105.003	1151.828
burst	50	0.5	X25519MLKEM768	1999	1	1	0.05	126.941	148.976	104.171	105.409	1118.865
burst	50	1	X25519	1991	9	9	0.44	168.011	357.797	103.267	405.821	1439.631
burst	50	1	MLKEM512	1997	3	3	0.15	157.472	275.809	103.162	361.891	1162.631
burst	50	1	MLKEM768	1998	2	2	0.10	149.410	206.288	103.379	363.923	1156.837
burst	50	1	MLKEM1024	1997	3	3	0.15	160.661	263.664	103.687	469.646	1170.640
burst	50	1	X25519MLKEM768	1999	1	1	0.05	156.513	261.805	104.128	410.157	1168.189
burst	50	2	X25519	1996	4	4	0.20	213.737	385.542	103.299	1120.557	2145.837
burst	50	2	MLKEM512	1990	11	11	0.54	213.239	421.548	103.145	1116.737	2092.184
burst	50	2	MLKEM768	1991	10	10	0.49	230.217	416.191	103.407	1129.733	2142.950
burst	50	2	MLKEM1024	1976	25	25	1.22	228.908	463.047	103.723	1127.012	2306.643
burst	50	2	X25519MLKEM768	1986	14	14	0.68	246.300	458.534	104.203	1145.990	2170.448
burst	50	5	X25519	1960	40	40	1.95	412.220	688.061	103.405	1487.556	3560.152
burst	50	5	MLKEM512	1952	49	49	2.39	404.953	697.883	103.264	1547.636	3582.408
burst	50	5	MLKEM768	1938	62	62	3.02	475.456	824.124	103.519	2069.035	4126.483
burst	50	5	MLKEM1024	1936	67	67	3.27	436.414	778.130	103.846	1963.463	4196.154
burst	50	5	X25519MLKEM768	1938	62	62	3.02	439.548	772.327	104.272	2037.834	4124.261
burst	100	0	X25519	125	0	0	0.00	203.301	0.344	203.318	203.745	203.941
burst	100	0	MLKEM512	125	0	0	0.00	203.151	0.354	203.180	203.497	203.813
burst	100	0	MLKEM768	125	0	0	0.00	203.439	0.328	203.454	203.844	204.047
burst	100	0	MLKEM1024	125	0	0	0.00	203.647	0.450	203.706	204.153	204.648
burst	100	0	X25519MLKEM768	125	0	0	0.00	204.171	0.491	204.175	204.808	205.123

burst	100	0.5	X25519	998	2	2	0.19	230.336	169.643	203.278	203.964	1251.110
burst	100	0.5	MLKEM512	999	1	1	0.10	232.731	228.946	203.141	203.743	1245.748
burst	100	0.5	MLKEM768	997	3	3	0.29	249.613	277.066	203.404	230.436	1259.158
burst	100	0.5	MLKEM1024	997	3	3	0.29	233.136	221.605	203.706	204.632	1232.658
burst	100	0.5	X25519MLKEM768	998	2	2	0.19	254.225	305.898	204.188	231.474	1254.436
burst	100	1	X25519	1000	0	0	0.00	251.967	265.865	203.308	508.275	1238.492
burst	100	1	MLKEM512	997	3	3	0.29	253.399	231.564	203.174	511.052	1274.984
burst	100	1	MLKEM768	998	2	2	0.19	269.072	339.667	203.420	717.861	1263.561
burst	100	1	MLKEM1024	1000	0	0	0.00	261.967	271.753	203.729	622.146	1252.869
burst	100	1	X25519MLKEM768	996	4	4	0.38	258.394	297.885	204.210	506.065	1252.873
burst	100	2	X25519	988	12	12	1.14	341.147	469.184	203.359	1226.952	2693.592
burst	100	2	MLKEM512	998	2	2	0.19	317.124	370.536	203.221	1232.444	1851.898
burst	100	2	MLKEM768	995	5	5	0.48	306.508	336.281	203.453	1214.230	1440.930
burst	100	2	MLKEM1024	979	21	21	2.00	358.148	502.981	203.772	1238.000	2709.239
burst	100	2	X25519MLKEM768	993	7	7	0.67	327.840	426.968	204.238	1235.931	1898.119
burst	100	5	X25519	982	18	18	1.71	531.930	724.267	203.480	1719.040	3757.368
burst	100	5	MLKEM512	973	27	27	2.57	542.716	752.861	203.326	2138.676	3923.706
burst	100	5	MLKEM768	976	24	24	2.29	555.830	760.946	203.562	2199.367	3846.993
burst	100	5	MLKEM1024	965	38	38	3.62	605.218	875.785	203.906	2369.981	4400.929
burst	100	5	X25519MLKEM768	975	25	25	2.38	546.328	745.793	204.366	2125.500	3664.425
burst	200	0	X25519	125	0	0	0.00	403.162	0.406	403.227	403.640	404.206
burst	200	0	MLKEM512	125	0	0	0.00	403.192	0.332	403.182	403.737	403.960
burst	200	0	MLKEM768	125	0	0	0.00	403.343	0.375	403.364	403.966	404.252
burst	200	0	MLKEM1024	125	0	0	0.00	403.678	0.369	403.710	404.289	404.604
burst	200	0	X25519MLKEM768	125	0	0	0.00	404.078	0.428	404.150	404.685	405.033
burst	200	0.5	X25519	993	7	7	0.67	456.257	284.137	403.309	455.979	1677.435
burst	200	0.5	MLKEM512	997	3	3	0.29	431.628	192.021	403.187	403.967	1476.706
burst	200	0.5	MLKEM768	994	6	6	0.57	438.794	210.713	403.394	404.525	1628.889
burst	200	0.5	MLKEM1024	997	3	3	0.29	438.964	228.733	403.723	404.672	1477.124
burst	200	0.5	X25519MLKEM768	997	3	3	0.29	440.267	256.107	404.186	405.099	1452.999
burst	200	1	X25519	994	6	6	0.57	477.829	314.788	403.319	1053.452	1708.941
burst	200	1	MLKEM512	998	2	2	0.19	494.859	449.303	403.179	1036.682	2876.990
burst	200	1	MLKEM768	998	3	3	0.29	483.339	364.178	403.429	1230.524	2073.520
burst	200	1	MLKEM1024	993	8	8	0.76	490.274	374.760	403.725	1416.261	1755.964
burst	200	1	X25519MLKEM768	995	5	5	0.48	482.515	382.017	404.171	1025.387	1656.971
burst	200	2	X25519	991	10	10	0.95	527.641	470.364	403.316	1419.175	2869.409
burst	200	2	MLKEM512	992	10	10	0.95	585.616	577.881	403.201	1470.399	3463.292
burst	200	2	MLKEM768	994	7	7	0.67	554.880	512.931	403.459	1453.313	2854.526

burst	200	2	MLKEM1024	991	9	9	0.86	576.643	545.023	403.796	1470.790	3045.753
burst	200	2	X25519MLKEM768	995	5	5	0.48	546.617	447.605	404.192	1454.593	2439.248
burst	200	5	X25519	966	34	34	3.24	813.394	824.842	403.444	2486.610	4573.936
burst	200	5	MLKEM512	965	37	37	3.52	812.403	818.902	403.309	2451.545	4413.316
burst	200	5	MLKEM768	978	22	22	2.10	767.164	715.492	403.597	2103.829	3492.460
burst	200	5	MLKEM1024	962	40	40	3.81	832.721	863.248	403.947	2666.436	4346.736
burst	200	5	X25519MLKEM768	952	48	48	4.57	838.680	857.919	404.363	2658.840	4488.686

B.3 증폭 실험 원시 출력

B.3.1 개괄 스윕 — X25519 (인증서 500~16,000 B, RTT 50ms)

```
certsize   client_send   server_recv   server_send   amp    p50(ms)   RTT 수
   500        2560B         2601B         1290B       0.50    3.71     1-RTT
  1000        2560B         3101B         1795B       0.70    3.71     1-RTT
  2000        2560B         4101B         2810B       1.10    3.72     1-RTT
  3000        2560B         5101B         3835B       1.50    3.73     1-RTT
  3300        2560B         5401B         4140B       1.62    3.74     1-RTT
  3400        2560B         5501B         7899B       3.09   54.8      2-RTT
  4000        2560B         6101B         8510B       3.32   54.8      2-RTT
  5000        2560B         7101B         9525B       3.72   54.9      2-RTT
  8000        2560B        10101B        12560B       4.90   54.9      2-RTT
 16000        2560B        18101B        20670B       8.07   55.0      2-RTT
```

B.3.2 개괄 스윕 — X25519MLKEM768 (RTT 50ms)

```
certsize   client_send   server_recv   server_send   amp    p50(ms)   RTT 수
   500        3840B         3881B         1975B       0.51    4.88     1-RTT
  2000        3840B         5381B         3490B       0.91    4.88     1-RTT
  4000        3840B         7381B         5510B       1.44    4.89     1-RTT
  5000        3840B         8381B         6525B       1.70    4.89     1-RTT
  5900        3840B         9281B         7438B       1.94    4.90     1-RTT
  6000        3840B         9381B        11760B       3.06   55.7      2-RTT
  8000        3840B        11381B        13780B       3.59   55.7      2-RTT
 16000        3840B        19381B        21890B       5.70   55.8      2-RTT
```

B.3.3 정밀 임계 스윕 — X25519 (100 B 간격, n=30)

```
certsize   server_send   amp     RTT 수     certsize   server_send   amp     RTT 수
  3000        3835B        1.50    1-RTT       3300        4140B        1.62    1-RTT
  3100        3937B        1.54    1-RTT       3350        4192B        1.64    1-RTT
  3200        4038B        1.58    1-RTT       3400        7899B        3.09    2-RTT ★
  3250        4089B        1.60    1-RTT       3500        8001B        3.13    2-RTT
[임계 확정: 3,300~3,400 B 사이. 3,350은 1-RTT, 3,400은 2-RTT]
```

B.3.4 정밀 임계 스윕 — X25519MLKEM768 (100 B 간격, n=30)

```
certsize   server_send   amp     RTT 수     certsize   server_send   amp     RTT 수
  5700        7238B        1.89    1-RTT       5900        7438B        1.94    1-RTT
  5800        7338B        1.91    1-RTT       5950        7489B        1.95    1-RTT
  5850        7389B        1.92    1-RTT       6000       11760B        3.06    2-RTT ★
[임계 확정: 5,900~6,000 B 사이. 5,950은 1-RTT, 6,000은 2-RTT]
```

B.3.5 RTT 비례 검증 (인증서 4,032 B 고정)

```
RTT(ms)    X25519 p50    PQC p50    격차     격차/RTT
   10         24.6          15.1       9.5       0.95
   50        104.8          55.6      49.2       0.98
  100        205.1         105.8      99.3       0.99
  200        403.2         205.4     197.8       0.99
[X25519는 2-RTT 구간(4,032>3,400), PQC는 1-RTT 구간(4,032<5,900)]
[관계식: PQC ≈ 1×RTT + 5ms,  X25519 ≈ 2×RTT + 5ms]
```

B.3.6 재현성 검증 (3회 독립 실행 × 100회, 인증서 5,400 B)

```
실행    n     2-RTT 발생   비율      비고
  1     100        1        1.0%     대부분 1-RTT, 꼬리 1%
  2     100        1        1.0%     재현됨
```

```
  3     100        1        1.0%     재현됨
[5,400 B는 PQC 임계(5,900) 직전. 1% 수준의 2-RTT 꼬리가 일관되게 관측]
[별도 관측된 5,609 B에서의 초기 105.7ms 값은 300회 재시행에서 미재현 — 이상치로 판정]
```

B.3.7 패딩 인과 검증 (X25519 고정, Initial 패킷 크기 조작)

```
initpkt   client_send   budget(×3)   관측 임계       비고
  1200       2400B         7200B       3,300~3,400 B
  1280       2560B         7680B       3,300~3,400 B   기본값
  1350       2700B         8100B       4,400~4,600 B   임계 이동 ★
  1450       2914B         8742B       4,400~4,600 B
[같은 알고리즘·같은 인증서. 패딩만 늘려도 임계가 이동 → 원인은 바이트 수]
[교환비 client_send : 임계이동 ≈ 1 : 3.8]
```

부록 C

산출물 목록

본 연구가 남긴 파일 목록이다. 코드는 부록 A 에 전문이 있고, 데이터는 엑셀에 조건별 원본이 있다.

분류	파일	내용
보고서	연구방향_변경보고.pdf/docx	계획 대비 변경 승인 요청 (4 쪽)
보고서	실험환경_명세서.docx	논문 방법론 절 원자료 (6 쪽)
보고서	FEC_인수인계.docx	FEC 구현 담당자용 (6 쪽)
보고서	본 종합 기술 문서	전 과정 기록
데이터	ML-KEM_1 단계_측정수치_v2.xlsx	크기·연산시간·패킷분할 (하이브리드 포함)
데이터	sweep1_results.xlsx	TLS 1 차 스윕 120 조건
데이터	sweep2_results.xlsx	TLS 2 차 스윕 250 조건 + z 검정
데이터	quic_amplification_v4.xlsx	증폭 실험 9 시트
코드	hct_v2.c	TLS 측정기
코드	run_v3.sh	TLS 스윕 스크립트
코드	qhct.go / qpkt.go	QUIC 측정·계측
코드	patch1/2, amp_sweep.sh	증폭 실험 도구
노트	선행연구노트_Sakk2026.docx	Sakk 외(2026) 정독 노트

— 문서 끝 —